



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт № 8 «Компьютерные науки и прикладная математика»

Образовательный центр Института № 8

Группа М8О-208СВ-24 Направление 01.04.02 Прикладная математика и информатика

Программа Продуктовая разработка и разработка программных систем

Квалификация: инженер-исследователь

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

На тему: Предотвращение взаимных блокировок автономных агентов методами централизованного планирования

Автор ВКР: _____ Бирюков Виктор Владимирович _____ (_____)

Научный руководитель: _____ Судаков Владимир Анатольевич _____ (_____)

Консультант: _____ (_____)

Консультант: _____ (_____)

Рецензент: _____ Осипов Владимир Петрович _____ (_____)

К защите допустить

И.о. директора Образовательного
центра Института № 8

_____ Крылов Сергей Сергеевич _____ (_____)

_____ 2026 г.

РЕФЕРАТ

Выпускная квалификационная работа состоит из 76 страниц, 15 рисунков, 4 таблиц, 31 использованного источника, 2 приложений.

ПОИСК ПУТИ В ГРАФЕ, МНОГОАГЕНТНОЕ ПЛАНИРОВАНИЕ, АВТОНОМНЫЕ АГЕНТЫ, АДАПТИВНЫЙ ПОИСК ПУТИ, РАЗРЕШЕНИЕ КОНФЛИКТОВ

Объектом исследования в данной работе является централизованное управление группой автономных агентов, движущихся по графу дорожной сети с узкими местами.

Цель работы — разработать методы локального разрешения конфликтов для глобального управления автономными агентами на графе дорожной сети с узкими местами и сопоставить их с классическими методами многоагентного планирования.

Для достижения поставленной цели были проведены исследования в области алгоритмов многоагентного планирования (CBS, CCBS, LaCAM, WHCA*), методов одноагентного поиска путей (A^* , SIPP) и методов разрешения конфликтов на узких участках дорожной сети.

Основное содержание работы состояло в разработке двух методов локального разрешения конфликтов (реактивный разъезд и система семафоров), адаптивной модификации A^* с эмпирической оценкой скоростей рёбер, а также модульного симулятора городской среды для воспроизводимого сравнения алгоритмов.

Основными результатами работы являются предложенные алгоритмы управления, программная среда симуляции и экспериментальное сравнение с оптимальным алгоритмом CCBS на фрагментах реальной городской карты, показавшее, что сочетание адаптивного A^* и локального разрешения конфликтов не уступает многоагентному планированию, а на типичных размерах задачи превосходит его.

Результаты разработки предназначены для использования при проектировании систем централизованного управления парками автономных доставочных агентов.

Использование результатов данной работы позволяет существенно снизить вычислительные затраты на согласование движения большого числа агентов при сохранении сопоставимого качества совместного движения.

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	5
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	6
ВВЕДЕНИЕ	7
1 ПОСТАНОВКА ЗАДАЧИ	9
1.1 Актуальность работы	9
1.2 Постановка задачи	9
1.3 Техническое задание	11
2 МЕТОДЫ ЦЕНТРАЛИЗОВАННОГО УПРАВЛЕНИЯ АВТОНОМНЫМИ АГЕНТАМИ	12
2.1 Описание исходных данных	12
2.2 Методы многоагентного планирования	14
2.2.1 Постановка задачи	14
2.2.2 Иерархический кооперативный А*	16
2.2.3 Поиск на основе конфликтов	19
2.2.4 Поиск на основе конфликтов в непрерывном времени	22
2.2.5 Многоагентное планирование с ленивым добавлением ограничений	25
2.3 Методы разрешения конфликтов	28
2.3.1 Разрешение конфликтов путем управления разъездом агентов	29
2.3.2 Предотвращение конфликтов при помощи глобальных ограничений	33
2.4 Другие методы управления	35
2.4.1 Адаптивное планирование с учётом наблюдаемой загруженности среды	35
2.5 Метрики	39
3 СИМУЛЯЦИЯ ПОВЕДЕНИЯ АВТОНОМНЫХ АГЕНТОВ В ГОРОДСКОЙ СРЕДЕ	41
3.1 Архитектура симулятора	41
3.2 Компоненты симулятора	44
3.2.1 Цикл симуляции	44
3.2.2 Продвижение агентов	45
3.2.3 Разрешение конфликтов	47
3.2.4 Распределение заказов	48

3.2.5	Построение глобальных маршрутов	49
3.2.6	Сбор статистик	51
3.2.7	Пользовательский интерфейс	52
3.3	Ограничения и допущения	53
3.4	Получение исходных данных	54
3.5	Используемые технологии	56
4	РЕЗУЛЬТАТЫ РАБОТЫ	58
4.1	Исходные данные	58
4.2	Используемые метрики	58
4.3	Проверка статистической значимости	60
4.4	Результаты работы	61
4.4.1	Выбор метода планирования для методов разрешения конфликтов	62
4.4.2	Сравнение методов разрешения конфликтов	65
4.4.3	Сравнение с CCBS	66
	ЗАКЛЮЧЕНИЕ	69
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	71
	ПРИЛОЖЕНИЕ А Графы дорожной сети	74
	ПРИЛОЖЕНИЕ Б Исходный код	76

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей выпускной квалификационной работе применяют следующие термины с соответствующими определениями:

Flowtime — суммарное время, затраченное всеми агентами на достижение целей

Makespan — момент времени, к которому все агенты достигают своих целей

Success rate — доля экземпляров задачи, решённых алгоритмом в пределах заданного бюджета времени

Агент — автономный исполнитель, перемещающийся по графу дорожной сети

Конфликт — ситуация, в которой согласно индивидуальным планам два или более агентов оказываются одновременно в недопустимой конфигурации

Маршрут (индивидуальный план) — последовательность вершин и действий агента, ведущая от его исходной к целевой вершине

Небезопасный интервал — максимальный интервал времени, в течение которого начало действия агента приводит к столкновению с действием другого агента

Ограничение — запрет агенту находиться в заданной вершине или выполнять заданное действие в указанный момент или интервал времени

Расписание — неубывающая последовательность плановых времён прохождения вершин маршрута

Эвристика — функция, оценивающая снизу стоимость оставшегося пути в алгоритмах информированного поиска

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящей выпускной квалификационной работе применяют следующие сокращения и обозначения:

CAT — conflict avoidance table, таблица предотвращения конфликтов

CBS — conflict-based search, поиск на основе конфликтов

CCBS — continuous-time conflict-based search, поиск на основе конфликтов в непрерывном времени

LaCAM — lazy constraints addition search for MAPF, поиск с ленивым добавлением ограничений

MAPF — multi-agent path finding, многоагентное планирование путей

PIBT — priority inheritance with backtracking, приоритетное наследование с возвратом

RHCR — rolling-horizon collision resolution, разрешение коллизий со скользящим горизонтом

RL — reinforcement learning, обучение с подкреплением

SIC — sum of individual costs, эвристика суммы индивидуальных стоимостей

SIPP — safe interval path planning, планирование путей по безопасным интервалам

SOC — sum of costs, суммарная стоимость решения

SOF — sum of fuel, суммарные перемещения без учёта ожиданий

WHCA* — windowed hierarchical cooperative A*, оконный иерархический кооперативный A*

ВВЕДЕНИЕ

Актуальность темы данной работы связана с бурным развитием систем автономной доставки на последней миле — наиболее затратном и наименее автоматизированном участке логистической цепочки. С теоретической точки зрения эти системы порождают задачу согласованного управления группой автономных агентов на графе дорожной сети с узкими местами, на которых одновременное встречное движение физически невозможно. Классические оптимальные алгоритмы многоагентного планирования плохо масштабируются на непрерывное время, продолжительный режим работы и сотни-тысячи агентов в реальной городской среде, в связи с чем актуальным является поиск более лёгких альтернативных подходов.

Таким образом, работа актуальна и с теоретической, и с практической точек зрения.

Цель работы — разработать методы локального разрешения конфликтов для глобального управления автономными агентами на графе дорожной сети с узкими местами и сопоставить их с классическими методами многоагентного планирования в задаче предотвращения взаимоблокировок.

Для достижения поставленной цели в работе были решены следующие задачи:

- сформулирована формальная модель задачи и набор интегральных метрик для оценки качества управления в продолжительном режиме работы;
- проведён обзор существующих методов многоагентного планирования (WHCA*, CBS, CCBS, LaCAM);
- предложены альтернативные методы управления, основанные на локальном разрешении конфликтов, а также адаптивная модификация A* на основе статистики скоростей рёбер;
- разработана модульная программная среда симуляции и подготовлен набор тестовых графов на основе фрагментов реальной карты Москвы;
- проведён сравнительный эксперимент с оценкой статистической значимости различий.

Работа основывалась на следующих инструментах и методах:

- алгоритмы поиска на графах (A*, SIPP) и многоагентного

планирования (CBS, CCBS);

- методы вычислительной геометрии для подготовки графов и обнаружения конфликтов;
- методы непараметрической статистики: U-критерий Манна–Уитни, дельта Клиффа, поправка Холма;
- язык программирования C++ и сопутствующие библиотеки для реализации симулятора; язык программирования Python для подготовки данных и статистического анализа.

Основными результатами, полученными в работе, являются:

- формальная постановка задачи и согласованный с ней набор интегральных метрик качества;
- два метода локального разрешения конфликтов на узких рёбрах и метод адаптивной маршрутизации на основе A^* с эмпирической оценкой скоростей рёбер;
- модульный симулятор городской среды и утилита подготовки графов из участка карты OpenStreetMap;
- экспериментальное сравнение с алгоритмом CCBS, показавшее, что сочетание адаптивного A^* и локального разрешения конфликтов не уступает многоагентному планированию, а на типичных размерах задачи превосходит его по всем интегральным метрикам.

Результаты работы предназначены для использования при проектировании систем централизованного управления парками автономных доставочных агентов.

Использование разработки позволяет существенно снизить вычислительные затраты на согласование движения большого числа агентов по сравнению с оптимальным многоагентным планированием при сохранении сопоставимого качества совместного движения.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Актуальность работы

В последнее десятилетие автономный транспорт перестал быть исключительно исследовательской темой и постепенно превратился в индустрию: появляются серийные автономные автомобили, беспилотные грузовики на закрытых маршрутах, а также мобильные роботы, обслуживающие складские и городские задачи. Особое место здесь занимает автономная доставка на последней миле — наиболее затратном и наименее автоматизированном участке логистической цепочки, где стоимость единицы перевозки традиционно велика, а график выполнения заказов жёстко ограничен ожиданиями получателя.

Развитие данного направления стимулировано сразу несколькими факторами: ростом объёмов электронной торговли, дефицитом курьерского труда в крупных городах и необходимостью снижать выбросы транспортных средств. В ответ на эти вызовы появилось значительное количество исследовательских и промышленных платформ, обзор которых приведён, например, в работе [1]. Несмотря на разнообразие технических решений, все они в той или иной форме сталкиваются с одним и тем же набором алгоритмических задач, связанных с безопасным и эффективным перемещением группы агентов в общей среде.

В настоящей работе фокус внимания направлен именно на задачу автономной доставки на последней миле и рассматриваются алгоритмические аспекты совместного движения автономных роботов в городской среде.

1.2 Постановка задачи

В контексте автономной доставки можно выделить два уровня планирования движения. Локальное планирование отвечает за поведение агента на коротком горизонте: реактивный обход препятствий, удержание полосы, учёт кинематических ограничений платформы, реакция на пешеходов и другие подвижные объекты. Глобальное планирование, напротив, оперирует на масштабе всей сети маршрутов и определяет, по каким участкам и в какой последовательности агенты должны проходить, чтобы выполнить поставленные транспортные задачи.

В последние годы задача локального планирования для автономных мобильных агентов активно развивается: наряду с классическими реактивными методами всё чаще применяются оптимизационные, learning-based и гибридные подходы [2; 3; 4]. Современные методы демонстрируют сдвиг от статического построения маршрутов к адаптивному локальному принятию решений в динамических и частично наблюдаемых средах и позволяют справляться с подавляющим большинством типовых ситуаций: расхождением со встречным агентом, объездом препятствия, ожиданием освобождения проезда.

Тем не менее существует класс ситуаций, в которых локального планирования принципиально недостаточно, — так, например, в городской (и не только) среде оптимальному перемещению агентов могут мешать узкие места. Под узким местом понимается участок дорожной сети, на котором физически невозможен одновременный проезд нескольких агентов, а решение о порядке прохождения требует учёта намерений всех заинтересованных агентов и состояния сети в целом. Чисто реактивный локальный планировщик, оперирующий только локальной информацией, в таких условиях склонен к взаимоблокировкам и неоптимальным разъездам. Естественным способом устранить эту проблему является глобальное планирование, явно согласующее маршруты и расписания агентов в узких местах.

В настоящей работе рассматривается задача глобального планирования движения группы автономных агентов по взвешенному графу дорожной сети, содержащему узкие места. Требуется для каждого агента определить маршрут и временной профиль его прохождения такие, чтобы согласованный план движения всех агентов гарантировал достижение каждым агентом его пункта назначения, не приводил к взаимоблокировкам в узких местах и минимизировал выбранный показатель качества. При этом рассматриваются два принципиально различных подхода к работе с узкими местами: с одной стороны — методы планирования, заранее учитывающие возможные конфликты при построении совместного плана; с другой — методы непосредственного разрешения конфликтов, в которых маршруты агентов строятся независимо, а согласование производится только в момент фактического соприкосновения интересов в узком месте. Поведение агентов на участках, где локального планирования достаточно, выносится за рамки

задачи и считается обеспеченным локальным планировщиком.

1.3 Техническое задание

В рамках поставленной цели работа предусматривает решение следующих задач:

- сформулировать формальную модель задачи глобального планирования для группы автономных доставочных агентов, движущихся по взвешенному графу дорожной сети с возможным наличием узких мест;
- изучить существующие методы глобального планирования, применимые к поставленной модели, и выделить их сильные и слабые стороны;
- предложить альтернативные методы глобального планирования, ориентированные на эффективную работу в условиях узких мест;
- сформулировать набор метрик для оценки методов глобального планирования;
- разработать программную среду, позволяющую воспроизводимо моделировать движение группы агентов и проверять корректность и качество предложенных методов;
- подготовить репрезентативный набор тестовых сценариев, включающий графы дорожных сетей с узкими местами различной структуры и конфигурации агентов разной плотности;
- провести сравнительный эксперимент, оценить предложенные методы относительно существующих и сформулировать рекомендации по их применению.

2 МЕТОДЫ ЦЕНТРАЛИЗОВАННОГО УПРАВЛЕНИЯ АВТОНОМНЫМИ АГЕНТАМИ

В данной главе будут рассмотрены различные методы планирования путей, а также методы централизованного разрешения конфликтов, возникающих в ходе непосредственного следования агентами по спланированным маршрутам.

2.1 Описание исходных данных

Исходные данные для задачи планирования представляют из себя две сущности — окружение, в котором перемещаются агенты, и сами агенты. Так как в рамках данной задачи интерес представляет глобальное поведение агентов, окружение можно представить в виде взвешенного графа из n вершин и m ребер:

$$G = (V, E, W), \quad (1)$$

где V — множество вершин $\{v_1, \dots, v_n\}$,

E — множество ребер ($e = \{v_i, v_j\}$),

$W : E \rightarrow \mathbb{R}$ — функция, задающая вес ребра.

В таком графе ребрам могут соответствовать улицы, отдельные участки улиц, комнаты или просто некоторые ограниченные части пространства.

Другой частью исходных данных считаем множество из k пронумерованных агентов:

$$A = \{1, \dots, k\}. \quad (2)$$

В такой постановке исходные данные этой задачи очень похожи на исходные данные задачи многоагентного планирования MAPF — Multi Agent Path Finding [5].

Исходные данные классической задачи многоагентного планирования представляют из себя граф $G = (E, V)$ и множество агентов A (2). Для каждого агента $i \in A$ определены начальная (стартовая) $s_i \in V$ и конечная (целевая) $g_i \in V$ вершины графа. Для классического MAPF характерно использование дискретного времени, что выражается в использовании

невзвешенного графа и допущениях о перемещении агентов, для которых любое действие (перемещение по ребру или ожидание) занимает фиксированное время.

Такая модель плохо подходит для описания реальных ситуаций, в которых разные действия могут иметь разную продолжительность, агенты перемещаются с переменной скоростью или дискретизация пространства приводит к чрезмерному увеличению размерности задачи. Поэтому для многоагентного планирования существует также формулировка [6], в которой используется взвешенный граф G (1). Вес ребра отражает его длину и агенты перемещаются по ребру в течении некоторого времени, которое определяется их скоростью.

Таким образом, в качестве исходных данных в работе будет использоваться взвешенный граф G (1) и множество агентов A (2). Так как речь идет об определенных окружениях для агентов, предполагается, что графы представляют из себя неориентированные планарные графы, вершины которых расположены на двумерной плоскости $V \subset \mathbb{R}^2$, а ребра — отрезки между соответствующими точками. В качестве веса ребра используется его длина:

$$\begin{aligned} W : E &\rightarrow \mathbb{R}_+ \\ W(e) &= \|e\|_2, \end{aligned} \tag{3}$$

где $e \in E$.

Помимо геометрических характеристик, каждому ребру сопоставляется признак, отражающий его пропускную способность. Часть рёбер графа соответствует достаточно широким участкам среды, на которых одновременное присутствие нескольких агентов и их встречное движение физически возможны без столкновений. Остальные рёбра соответствуют узким участкам, на которых в каждый момент времени допустимо движение агентов только в одном направлении. Это разделение задаётся вместе с исходным графом в виде разбиения множества рёбер:

$$E = E_w \sqcup E_n, \tag{4}$$

где E_w — множество широких рёбер,

E_n — множество узких рёбер.

2.2 Методы многоагентного планирования

MARF — важный класс задач многоагентного планирования, в котором требуется построить маршруты для нескольких агентов с ключевым ограничением: агенты должны иметь возможность следовать по этим маршрутам одновременно, не сталкиваясь друг с другом. MARF находит широкое применение в автоматизированных складах, автономных транспортных средствах и робототехнике.

2.2.1 Постановка задачи

Постановка классической задачи MARF выглядит следующим образом. Исходные данные описаны в предыдущем разделе и представляют из себя граф и множество агентов. Предполагается, что время дискретно: на каждом шаге каждый агент находится в одной из вершин графа и выполняет ровно одно действие. Действие в классической задаче MARF — это функция $a : V \rightarrow V$, где $a(v) = v'$ означает, что агент, находящийся в вершине v и выполняющий действие a , окажется в вершине v' на следующем шаге. Каждый агент может выполнять два типа действий: ожидание и перемещение. Ожидание означает, что агент остаётся в текущей вершине ещё один шаг, перемещение — что агент переходит из текущей вершины v в смежную вершину v' графа.

Для последовательности действий $\pi = (a_1, \dots, a_n)$ и агента i обозначим через $\pi_i[x]$ положение агента после выполнения первых x действий из π , начиная из стартовой вершины s_i :

$$\pi_i[x] = a_x(a_{x-1}(\dots a_1(s_i))). \quad (5)$$

Последовательность действий π является индивидуальным планом агента i тогда и только тогда, когда выполнение этой последовательности из s_i приводит агента в целевую вершину g_i , то есть $\pi_i[|\pi|] = g_i$. Решением задачи называется набор из k индивидуальных планов — по одному для каждого агента.

Основная цель алгоритмов решения MARF — найти решение, которое может быть выполнено без столкновений. Для этого в процессе планирования

используется понятие конфликта: решение называется допустимым тогда и только тогда, когда между любыми двумя индивидуальными планами нет конфликтов. Определение конфликта зависит от модели окружения, обычно выделяют несколько видов конфликтов (π_i и π_j — пара индивидуальных планов) [5]:

- **Конфликт в вершине** возникает между π_i и π_j , когда согласно этим планам агенты оказываются в одной вершине в один и тот же момент времени. То есть существует шаг x такой, что $\pi_i[x] = \pi_j[x]$.
- **Конфликт в ребре** возникает между π_i и π_j , когда агенты проходят по одному и тому же ребру в одном направлении на одном и том же шаге. То есть существует шаг x такой, что $\pi_i[x] = \pi_j[x]$ и $\pi_i[x + 1] = \pi_j[x + 1]$.
- **Конфликт следования** возникает между π_i и π_j , когда один агент занимает вершину, которую другой агент покинул на предыдущем шаге. То есть существует шаг x такой, что $\pi_i[x + 1] = \pi_j[x]$.
- **Циклический конфликт** возникает между набором планов $\pi_i, \pi_{i+1}, \dots, \pi_j$, когда на одном и том же шаге каждый агент перемещается в вершину, которую только что покинул другой агент. То есть существует шаг x такой, что $\pi_i[x + 1] = \pi_{i+1}[x]$, $\pi_{i+1}[x + 1] = \pi_{i+2}[x]$, $\dots$, $\pi_{j-1}[x + 1] = \pi_j[x]$, $\pi_j[x + 1] = \pi_i[x]$.
- **Конфликт обмена** возникает между π_i и π_j , когда агенты меняются местами за один шаг. То есть существует шаг x такой, что $\pi_i[x] = \pi_j[x + 1]$ и $\pi_j[x] = \pi_i[x + 1]$.

Использование дискретного времени ограничивает применимость методов решения классической задачи. Для возможности работы с непрерывным временем используется более сложная модель, в которой граф, описывающий возможные положения агентов, является взвешенным. В таком случае различные действия могут иметь различную продолжительность: время перемещение агента зависит от его скорости, время ожидания неограниченно и так далее.

В условиях непрерывного времени не имеет смысла разделение конфликтов на категории, так как конфликт может произойти в любой момент времени между агентами, находящимися в процессе выполнения своих действий. Вместо этого, например, может быть использовано определение конфликтов на основе пересечения геометрических форм агентов [6].

Другой особенностью классической (даже непрерывной) задачи является то, что она представляет из себя только «одноразовый» вариант фактической проблемы во многих приложениях. Обычно после того, как агент достигает своего целевого местоположения, он не останавливается и не ждет там вечно. Вместо этого ему назначается новое целевое местоположение, и от него требуется продолжать двигаться. Такой вариант называется продолжительным (lifelong) MAPF [7] и характеризуется постоянным назначением агентам новых целевых местоположений.

2.2.2 Иерархический кооперативный A^*

Алгоритм оконного иерархического кооперативного A^* (Windowed Hierarchical Cooperative A^* , WHCA*) [8] относится к приоритетным декомпозиционным методам решения задачи многоагентного планирования. Ключевая идея таких методов состоит в том, что совместная задача не решается в полном пространстве состояний всех агентов. Вместо этого агенты планируются последовательно: для каждого агента строится индивидуальный путь, а уже построенные пути считаются динамическими препятствиями для последующих агентов. Такой подход не гарантирует оптимальность и полноту в общем случае, но резко уменьшает размерность поиска и поэтому хорошо подходит для задач, где требуется получать допустимые маршруты за ограниченное время.

Базовым алгоритмом для WHCA* является кооперативный A^* (Cooperative A^* , CA*). В CA* одноагентный поиск выполняется не в обычном графе G , а в пространственно-временном графе. Состояние имеет вид (v, t) , где $v \in V$ — вершина исходного графа, а t — дискретный момент времени. Из состояния (v, t) агент может перейти в состояние $(v', t + 1)$, если v' смежна с v , либо остаться на месте, перейдя в $(v, t + 1)$. Действие ожидания необходимо, так как иногда агенту выгоднее пропустить другого агента, чем перестраивать весь маршрут.

Для передачи информации между одноагентными поисками используется таблица резервирования (reservation table). После того как путь очередного агента найден, все занятые им состояния (v, t) заносятся в таблицу и считаются недоступными для последующих агентов. Тем самым планирование следующего агента сводится к обычному поиску A^* в пространстве (v, t) с дополнительным условием: состояние запрещено, если

оно уже зарезервировано другим агентом. В более точных реализациях таблица резервирования может также запрещать прохождение одного ребра в противоположных направлениях на одном шаге, что предотвращает конфликт обмена.

Основной недостаток CA^* состоит в том, что поиск ведётся в трёхмерном пространстве и должен построить полный путь до цели. Кроме того, простая эвристика, например манхэттенское расстояние на сетке, плохо учитывает препятствия и приводит к большому числу раскрытых состояний. Иерархический кооперативный A^* (Hierarchical Cooperative A^* , HCA^*) улучшает CA^* за счёт более точной эвристики. Для этого вводится абстрактное пространство, совпадающее с исходным графом без временного измерения и без учёта других агентов. Расстояние от вершины v до цели g_i в таком абстрактном графе является нижней оценкой стоимости оставшегося пути в пространственно-временном графе, так как взаимодействие с другими агентами может только увеличить длину пути, но не уменьшить её.

Чтобы не пересчитывать абстрактные расстояния с нуля при каждом запросе, в HCA^* используется обратный возобновляемый A^* (Reverse Resumable A^* , RRA^*). Поиск запускается из целевой вершины g_i в обратном направлении и продолжается до тех пор, пока не будет раскрыта вершина, расстояние до которой требуется оценить. Если позднее потребуется расстояние для другой вершины, уже построенные множества OPEN и CLOSED не удаляются: поиск просто возобновляется. После раскрытия вершины v значение $g(v)$ в обратном поиске равно точному расстоянию от v до g_i в абстрактном графе и может использоваться как эвристика нижнего уровня.

$WHCA^*$ добавляет к этой схеме оконное планирование. Вместо построения полного пространственно-временного пути до цели агент планирует только первые w шагов, где w — размер окна. Внутри этого окна учитываются таблица резервирования и маршруты других агентов, а за пределами окна взаимодействия игнорируются и стоимость оставшегося пути оценивается абстрактной эвристикой HCA^* . Иными словами, $WHCA^*$ рассматривает точную кооперативную задачу только на ближайшем горизонте, а дальнюю часть маршрута заменяет оценкой расстояния до цели.

Такой приём можно описать как добавление специальных терминальных переходов. Пусть во время поиска агент достиг состояния

(v, w) на границе окна. Тогда из него вводится переход непосредственно в цель с весом, равным абстрактному расстоянию $\text{dist}_{abs}(v, g_i)$, найденному при помощи RRA*. Поиск A* выбирает частичный маршрут, минимизирующий сумму стоимости первых w действий и оценки оставшегося пути. При этом фактически исполняется только начальная часть найденного маршрута; затем окно сдвигается вперёд, таблица резервирования обновляется и планирование повторяется.

Псевдокод WHCA* приведён на рисунке 1. В нём предполагается, что агенты обрабатываются в некотором порядке приоритетов. Этот порядок может быть фиксированным, но на практике его часто изменяют между окнами: тогда каждый агент периодически получает высокий приоритет, что уменьшает зависимость алгоритма от неудачного начального упорядочивания.

Входные данные: Экземпляр задачи MAPF, размер окна w

Результат: Набор частичных маршрутов агентов

- 1 $RT \leftarrow \emptyset$ — таблица резервирования;
- 2 выбрать порядок агентов;
- 3 **для каждого** агент a_i **в выбранном порядке** **выполнять**
 - 4 построить или возобновить RRA* из цели g_i ;
 - 5 $\pi_i \leftarrow A^*$ в пространстве (v, t) на глубину не более w ;
 - 6 при раскрытии состояния (v, w) использовать стоимость $\text{dist}_{abs}(v, g_i)$;
 - 7 добавить первые w шагов пути π_i в RT ;
- 8 выполнить начальную часть построенных маршрутов;
- 9 сдвинуть окно вперёд и повторить планирование при необходимости;

Рисунок 1 – Оконный иерархический кооперативный A*

Оконное планирование решает сразу несколько практических проблем. Во-первых, агент не обязан резервировать весь путь до цели, поэтому другой агент может позднее занять освобождённое пространство, а достигший цели агент при необходимости может отойти и пропустить остальных. Во-вторых, вычисления распределяются по времени: при пересчёте маршрута только на ближайшие w шагов стоимость одной итерации существенно меньше, чем при

полном планировании до цели. В-третьих, регулярный пересчёт маршрутов делает алгоритм пригодным для динамических сред, где цели или препятствия могут изменяться во время движения.

Размер окна является основным параметром WHCA*. При большом w алгоритм становится ближе к HCA*: он учитывает больше будущих взаимодействий и строит более согласованные маршруты, но требует больше времени на один поиск. При малом w алгоритм становится ближе к локальному перепланированию: он быстрее реагирует на изменения, но хуже разрешает длинные конфликты в узких проходах. Поэтому окно обычно выбирают сопоставимым с характерной длительностью прохождения таких проходов группой агентов.

WHCA* не является оптимальным алгоритмом MAPF. Его результат зависит от порядка агентов, размера окна и выбранной эвристики; возможны ситуации, когда ранняя резервация пути одним агентом делает задачу для другого агента неразрешимой, хотя совместное решение существует. Тем не менее сочетание пространственно-временного поиска, таблицы резервирования, иерархической эвристики и оконного перепланирования делает WHCA* эффективным практическим методом для больших динамических сред, где полные оптимальные алгоритмы оказываются слишком дорогими.

2.2.3 Поиск на основе конфликтов

Алгоритм поиска на основе конфликтов (Conflict-Based Search, CBS) является двухуровневым алгоритмом оптимального решения задачи MAPF. Ключевая идея CBS состоит в декомпозиции многоагентной задачи на набор одноагентных задач поиска пути: вместо того чтобы искать решение в совместном пространстве состояний всех агентов, алгоритм строит пути для каждого агента независимо, а возникающие конфликты разрешает путём добавления ограничений.

В CBS используется следующий набор понятий. Ограничение для агента a_i — тройка (a_i, v, t) , запрещающая ему находиться в вершине v в момент времени t . Путь агента a_i называется согласованным, если он удовлетворяет всем его ограничениям; набор из k таких путей образует согласованное решение. Конфликт — четвёрка (a_i, a_j, v, t) , фиксирующая одновременное нахождение агентов a_i и a_j в вершине v в момент t .

Согласованное решение, не содержащее конфликтов, называется допустимым.

Верхний уровень CBS выполняет поиск в дереве ограничений (Constraint Tree, CT) — бинарном дереве, каждый узел N которого содержит три поля:

- $N.constraints$ — накопленное множество ограничений (корень содержит пустое множество, каждый дочерний узел наследует ограничения родителя и добавляет одно новое ограничение для одного агента);
- $N.solution$ — набор из k путей, по одному для каждого агента, причём путь агента a_i согласован с его ограничениями в узле N (пути находятся нижним уровнем алгоритма);
- $N.cost$ — суммарная стоимость решения (сумма длин индивидуальных путей), используемая как f -значение узла при поиске.

Узел N является целевым, если $N.solution$ допустимо, то есть ни один из путей не содержит конфликтов.

Верхний уровень CBS реализует поиск по первому наилучшему совпадению в дереве CT. Алгоритм инициализируется корневым узлом с пустым множеством ограничений; нижний уровень строит оптимальный индивидуальный путь для каждого агента без учёта остальных. Стоимость корня вычисляется как сумма индивидуальных стоимостей (эвристика SIC — Sum of Individual Costs).

На каждой итерации из очереди OPEN извлекается узел P с наименьшей стоимостью. Набор путей $P.solution$ проверяется на наличие конфликтов путём симуляции совместного движения агентов. Если конфликтов нет — узел объявляется целевым и его решение возвращается. В противном случае фиксируется первый найденный конфликт $C = (a_i, a_j, v, t)$.

Поскольку в любом допустимом решении не более одного из агентов a_i , a_j может находиться в вершине v в момент t , хотя бы одно из ограничений (a_i, v, t) или (a_j, v, t) должно присутствовать в любом допустимом решении. Для гарантии оптимальности CBS рассматривает оба варианта: порождаются два дочерних узла, каждый из которых наследует ограничения P и добавляет одно из двух ограничений. Оба узла помещаются в OPEN.

Псевдокод верхнего уровня CBS приведён на рисунке 2.

Входные данные: Экземпляр задачи MAPF

Результат: Оптимальное допустимое решение

```
1  $R.constraints \leftarrow \emptyset$ ;  
2  $R.solution \leftarrow$  нижний уровень для всех агентов;  
3  $R.cost \leftarrow SIC(R.solution)$ ;  
4 добавить  $R$  в OPEN;  
5 до тех пор, пока OPEN  $\neq \emptyset$  выполнять  
6    $P \leftarrow$  узел с наименьшей стоимостью из OPEN;  
7   проверить пути в  $P.solution$  на конфликты;  
8   если конфликтов нет тогда  
9     возвратить  $P.solution$ ;  
10   $C \leftarrow (a_i, a_j, v, t)$  — первый конфликт в  $P$ ;  
11  для каждого агента  $a \in \{a_i, a_j\}$  выполнять  
12     $A \leftarrow$  новый узел;  
13     $A.constraints \leftarrow P.constraints \cup \{(a, v, t)\}$ ;  
14     $A.solution \leftarrow P.solution$ ;  
15    обновить путь агента  $a$  в  $A.solution$  вызовом нижнего уровня;  
16     $A.cost \leftarrow SIC(A.solution)$ ;  
17    добавить  $A$  в OPEN;
```

Рисунок 2 – Верхний уровень CBS

Нижний уровень CBS получает на вход агента a_i и множество его ограничений. Задача нижнего уровня — найти оптимальный путь для a_i , удовлетворяющий всем ограничениям, при этом остальные агенты полностью игнорируются. Поиск ведётся в трёхмерном пространстве состояний: две пространственные координаты и время. В качестве алгоритма поиска используется A^* с совершенной эвристикой по двум пространственным измерениям; состояние (x, t) отбрасывается, если существует ограничение (a_i, x, t) . Для улучшения качества решений на нижнем уровне применяется таблица предотвращения конфликтов (Conflict Avoidance Table, CAT): при равенстве f -значений двух состояний предпочтение отдаётся тому, которое порождает меньше конфликтов с путями остальных агентов.

CBS гарантирует оптимальность возвращаемого решения. Стоимость

узла N является нижней оценкой минимальной стоимости любого допустимого решения, согласованного с ограничениями N : пути, найденные нижним уровнем, оптимальны для каждого агента в отдельности, поэтому ни один агент не может достичь цели дешевле. Поскольку верхний уровень реализует поиск по первому наилучшему совпадению, первый найденный целевой узел имеет минимально возможную стоимость.

Пространство поиска верхнего уровня CBS экспоненциально зависит от числа конфликтов, а не от числа агентов, как в случае глобального A^* . Это делает CBS особенно эффективным в средах с узкими проходами, где число конфликтов невелико. В открытых пространствах, где конфликты возникают повсеместно, CBS может уступать глобальным методам поиска.

2.2.4 Поиск на основе конфликтов в непрерывном времени

Алгоритм CBS, описанный выше, предполагает дискретное время и единичную продолжительность всех действий. Большинство существующих алгоритмов MAPF разделяют эти ограничения: они работают на сетках с 4-связной топологией, предполагают, что каждое действие занимает ровно один шаг, и не учитывают геометрическую форму агентов. Алгоритм поиска на основе конфликтов в непрерывном времени (Continuous-time Conflict-Based Search, CCBS) [6] является первым алгоритмом MAPF, который одновременно поддерживает непрерывное время, произвольную продолжительность действий, произвольные графы (не только сетки) и агентов с геометрической формой, оставаясь при этом полным и оптимальным. Именно это сочетание свойств делает CCBS наиболее подходящим алгоритмом для рассматриваемой в данной работе задачи, в которой агенты перемещаются по взвешенному графу в непрерывном времени.

CCBS следует той же двухуровневой схеме, что и CBS, и отличается от него в трёх ключевых аспектах: определении конфликта, форме ограничений и алгоритме нижнего уровня.

Конфликт в CCBS определяется на уровне пар действий. Поскольку агенты могут иметь произвольную геометрическую форму и выполнять действия произвольной продолжительности, конфликты могут возникать между агентами, проходящими по разным рёбрам, а также между агентом, движущимся по ребру, и агентом, ожидающим в вершине. Формально, конфликт CCBS — это четвёрка $\langle a_i, t_i, a_j, t_j \rangle$, означающая, что если агент i

начинает выполнять действие a_i в момент t_i , а агент j — действие a_j в момент t_j , то они столкнутся. Обнаружение таких конфликтов требует геометрически точного механизма определения столкновений. CCBS как алгоритм не зависит от конкретного механизма обнаружения столкновений: для агентов в форме диска, движущихся с постоянной скоростью, существует замкнутая формула; в общем случае применяются методы вычислительной геометрии.

Ограничение в CCBS — это тройка $\langle i, a_i, [t_i, t_i^u] \rangle$, запрещающая агенту i начинать действие a_i в любой момент из интервала $[t_i, t_i^u]$. Интервал $[t_i, t_i^u]$ называется небезопасным интервалом действия a_i относительно a_j : это максимальный временной интервал, начиная с t_i , в течение которого выполнение a_i приводит к столкновению с a_j , выполняемым в момент t_j .

Верхний уровень CCBS работает аналогично CBS: выполняет поиск по первому наилучшему совпадению в дереве ограничений. На каждой итерации из очереди OPEN извлекается узел N с наименьшей суммарной стоимостью совместного плана (Sum of Costs, SOC). Механизм обнаружения столкновений проверяет, является ли N целевым узлом. Если нет, верхний уровень выбирает один из конфликтов $\langle a_i, t_i, a_j, t_j \rangle$ в N .П и порождает два дочерних узла N_i и N_j . Для этого CCBS вычисляет небезопасный интервал каждого действия относительно другого: в N_i добавляется ограничение, запрещающее агенту i выполнять a_i в его небезопасном интервале, в N_j — аналогичное ограничение для агента j .

Нижний уровень CCBS основан на алгоритме SIPP (Safe Interval Path Planning) [9] — одноагентном алгоритме поиска пути, разработанном для работы в непрерывном времени с движущимися препятствиями. SIPP вычисляет для каждой вершины $v \in V$ множество безопасных интервалов, где безопасный интервал — это максимальный непрерывный временной промежуток, в течение которого агент может находиться в v без столкновения с движущимися препятствиями. Максимальность означает, что расширение интервала в любую сторону приводит к столкновению. Ключевое наблюдение SIPP состоит в том, что число безопасных интервалов конечно и, как правило, невелико. Поиск ведётся в пространстве пар (вершина, безопасный интервал) с помощью A^* , что обеспечивает полноту и оптимальность.

Для учёта ограничений CCBS алгоритм SIPP модифицируется следующим образом. Пусть задано ограничение $\langle i, a_k, [t_k, t_k^u] \rangle$ для агента i . Рассматриваются два случая в зависимости от типа действия a_k .

Если a_k — действие перемещения из вершины v в вершину v' : агент не может начать это перемещение в интервале $[t_k, t_k^u)$. Если агент прибывает в v в момент $t \in [t_k, t_k^u)$, то действие перемещения из v в v' в момент t заменяется на составное действие: ожидание в v до момента t_k^u с последующим перемещением в v' .

Если a_k — действие ожидания в вершине v : агент не может ожидать в v в интервале $[t_k, t_k^u)$. Безопасный интервал вершины v , содержащий этот промежуток, разбивается на два: $[0, t_k]$ и $[t_k^u, \infty)$, что исключает ожидание в запрещённый период.

Модифицированный SIPP возвращает оптимальный путь, удовлетворяющий всем ограничениям CCBS. Пара ограничений, порождаемых при разрешении конфликта $\langle a_i, t_i, a_j, t_j \rangle$, является корректной: в любом оптимальном решении хотя бы одно из них выполняется. Это свойство вместе с оптимальностью нижнего уровня и поиском по первому наилучшему совпадению на верхнем уровне обеспечивает оптимальность и полноту CCBS в целом.

Практическая сложность CCBS связана с вычислением небезопасных интервалов. В отличие от обнаружения столкновений, для которого в ряде случаев существуют замкнутые формулы, вычисление небезопасного интервала в общем случае требует итеративного применения механизма обнаружения столкновений: начиная с момента t_i , время увеличивается на малый шаг $\Delta > 0$ до тех пор, пока столкновение не перестает обнаруживаться. Такой подход может давать завышенную оценку небезопасного интервала, что сохраняет корректность алгоритма, но может снижать качество решения.

Для ускорения обнаружения конфликтов в CCBS применяется гибридная эвристика. На первом этапе обнаруживаются все конфликты и выбираются кардинальные — те, разрешение которых гарантированно увеличивает суммарную стоимость решения. Если кардинальных конфликтов нет, для узлов поддеревья ниже текущего применяется эвристика прошлых конфликтов: пары агентов, конфликтовавшие ранее, проверяются в первую очередь, а поиск конфликтов прекращается при обнаружении первого из них. Такой гибридный подход сочетает преимущества быстрого обнаружения конфликтов и интеллектуального выбора конфликта для разрешения.

CCBS является первым алгоритмом MAPF, обеспечивающим оптимальность и полноту в постановке с непрерывным временем,

произвольной продолжительностью действий и агентами, имеющими геометрическую форму. В отличие от дискретных методов, CCBS применим к произвольным графам и не привязан к сеточным доменам, что делает его наиболее подходящим инструментом для задач планирования в реальных окружениях.

2.2.5 Многоагентное планирование с ленивым добавлением ограничений

Алгоритм многоагентного планирования с ленивым добавлением ограничений (Lazy Constraints Addition Search for MAPF, LaCAM) [10; 11; 12] является быстрым алгоритмом планирования для классической задачи MAPF. В отличие от CBS, где верхний уровень ищет ограничения, а нижний уровень строит индивидуальные пути агентов, LaCAM меняет смысл уровней поиска: верхний уровень ищет последовательность совместных конфигураций всех агентов, а нижний уровень лениво порождает ограничения, задающие возможную следующую конфигурацию.

Конфигурацией в LaCAM называется кортеж положений всех агентов, $X = (x_1, \dots, x_k)$, $x_i \in V$, где x_i — вершина, в которой находится агент i . Стартовая конфигурация имеет вид $S = (s_1, \dots, s_k)$, целевая — $G = (g_1, \dots, g_k)$. Две конфигурации X и Y считаются смежными, если каждый агент либо остаётся в своей вершине, либо переходит в смежную вершину, а переход из X в Y не содержит вершинных конфликтов и конфликтов обмена. Тогда решение MAPF можно понимать как путь в графе конфигураций от S до G .

Прямой поиск в графе конфигураций имеет слишком большое ветвление. Если максимальная степень исходного графа равна Δ , то из одной конфигурации потенциально может быть порождено порядка $O((\Delta + 1)^k)$ следующих конфигураций: каждый агент независимо выбирает одну из соседних вершин или ожидание. Именно эту проблему LaCAM решает ленивым порождением наследников. Вместо того чтобы сразу строить все возможные следующие конфигурации, алгоритм сначала пытается получить одну перспективную конфигурацию, а остальные варианты раскрывает только при необходимости.

Узел верхнего уровня N содержит конфигурацию $N.config$, дерево ограничений $N.tree$ и ссылку на родительский узел. Множество OPEN хранит

ещё не обработанные узлы верхнего уровня; в базовом варианте оно реализуется как стек, поэтому поиск ведётся в глубину. Если из OPEN извлечён узел с целевой конфигурацией G , решение восстанавливается обратным проходом по родительским ссылкам.

Нижний уровень LaSAM связан не с поиском индивидуальных путей, а с построением ограничений на следующую конфигурацию. Узлы нижнего уровня образуют дерево. Каждый узел, кроме корня, хранит положительное ограничение вида (a_i, v) , означающее, что в следующей конфигурации агент a_i должен находиться в вершине v . Путь от узла дерева к корню задаёт набор таких ограничений для нескольких агентов. При раскрытии узла нижнего уровня выбирается следующий агент, для которого на этом пути ещё нет ограничения, и порождаются ограничения на все допустимые варианты его следующего положения: соседние вершины и ожидание в текущей вершине.

После выбора узла верхнего уровня N и очередного узла нижнего уровня C вызывается процедура порождения конфигурации. Она должна построить конфигурацию Q , смежную с $N.config$ и удовлетворяющую всем ограничениям, заданным путём от C к корню дерева. Если такой конфигурации найти не удалось, текущая итерация завершается без добавления нового узла. Если конфигурация Q получена и ранее не встречалась, для неё создаётся новый узел верхнего уровня, который помещается в OPEN. Если же Q уже была рассмотрена, она не добавляется повторно.

Псевдокод LaSAM приведён на рисунке 3. Для краткости процедура `GetNewConfig` оставлена как внешняя: на практике в этой роли может использоваться алгоритм, который быстро строит одношаговый совместный переход, например приоритетное планирование с горизонтом в один шаг или PIBT [13].

Входные данные: Экземпляр задачи MAPF со стартовой конфигурацией S и целевой конфигурацией G

Результат: Допустимое решение или сообщение об отсутствии решения

```
1  $N_0.config \leftarrow S$ ;  
2  $N_0.tree \leftarrow$  дерево с корневым узлом без ограничения;  
3  $N_0.parent \leftarrow \perp$ ;  
4 добавить  $N_0$  в OPEN и Explored;  
5 до тех пор, пока OPEN  $\neq \emptyset$  выполнять  
6    $N \leftarrow$  верхний узел из OPEN;  
7   если  $N.config = G$  тогда  
8     возвратить восстановить путь по родительским ссылкам;  
9   если  $N.tree$  не содержит нераскрытых узлов тогда  
10    удалить  $N$  из OPEN и перейти к следующей итерации;  
11     $C \leftarrow$  очередной узел нижнего уровня из  $N.tree$ ;  
12    расширить  $C$  ограничениями для следующего агента;  
13     $Q \leftarrow \text{GetNewConfig}(N.config, C)$ ;  
14    если  $Q = \perp$  или  $Q \in \text{Explored}$  тогда  
15      перейти к следующей итерации;  
16    создать узел  $N'$  с конфигурацией  $Q$  и пустым деревом  
    ограничений;  
17     $N'.parent \leftarrow N$ ;  
18    добавить  $N'$  в OPEN и Explored;  
19 возвратить решение отсутствует;
```

Рисунок 3 – Многоагентное планирование с ленивым добавлением ограничений

Ленивость алгоритма проявляется в том, что верхний узел не удаляется сразу после первой попытки породить наследника. Он остаётся в OPEN до тех пор, пока его нижнее дерево ограничений не будет полностью раскрыто. Поэтому при повторном выборе того же верхнего узла LaSAM продолжает низкоуровневый поиск и генерирует новые ограничения, тем самым получая следующие возможные конфигурации. После полного раскрытия нижнего

дерева все конфигурации, смежные с $N.config$, уже были либо порождены, либо признаны невозможными, и верхний узел можно удалить.

Полнота LaSAM следует из конечности пространства поиска. Число возможных конфигураций конечно и не превосходит $|V|^k$. Для каждой конфигурации нижний уровень в конечном итоге перебирает все наборы одношаговых положений агентов, то есть все потенциальные смежные конфигурации. Следовательно, если существует путь от S к G в графе конфигураций, то алгоритм рано или поздно рассмотрит все его вершины и найдёт решение. Если такого пути нет, после исчерпания всех достижимых конфигураций алгоритм корректно сообщает об отсутствии решения.

Качество и скорость LaSAM существенно зависят от процедуры `GetNewConfig`. Если она часто возвращает конфигурации, приближающие агентов к целям, эффективное ветвление верхнего уровня становится малым: вместо полного перебора $O((\Delta + 1)^k)$ вариантов алгоритм долго движется по одной перспективной ветви и добавляет альтернативы только в сложных местах. В экспериментальной реализации LaSAM для этого использовался PIBT, что позволило решать задачи с сотнями и тысячами агентов за малое время. С другой стороны, в узких коридорах и других конфигурациях с сильной взаимной блокировкой алгоритм может вынужденно раскрывать большое число похожих конфигураций, что ухудшает практическую производительность.

Таким образом, LaSAM занимает промежуточное положение между полным поиском в совместном пространстве и быстрыми эвристическими методами. Он сохраняет полноту, так как ленивое добавление ограничений в итоге перебирает все допустимые одношаговые переходы, но на практике старается рассматривать только малую часть этого пространства. В отличие от оптимальных методов, базовый LaSAM не гарантирует минимальность суммарной стоимости решения, зато хорошо масштабируется по числу агентов и подходит для задач, где важнее быстро получить допустимый план.

2.3 Методы разрешения конфликтов

Описанные выше методы многоагентного планирования строят индивидуальные маршруты агентов с учётом возможных конфликтов уже на этапе поиска, что в общем случае требует значительных вычислительных ресурсов и существенно усложняет реализацию, особенно в условиях

непрерывного времени и продолжительного режима работы.

На практике зачастую есть возможность использовать механизмы, способные непосредственно воздействовать на движение агентов в процессе исполнения. То есть конфликты могут быть исключены и без учёта взаимодействий на этапе планирования. Это позволяет использовать простые и быстрые одноагентные алгоритмы поиска пути, перенося задачу согласования движения на отдельный уровень управления.

Качество совместного движения, получаемого в результате такого разделения обязанностей, может оказаться ниже, чем у решений, построенных полноценными многоагентными алгоритмами. Тем не менее экономия вычислительных ресурсов и простота реализации нередко перевешивают потери в оптимальности, особенно при большом числе агентов или частом перестроении маршрутов. В этом разделе рассматриваются методы, реализующие именно такой подход к разрешению конфликтов.

2.3.1 Разрешение конфликтов путем управления разъездом агентов

Наиболее прямолинейный подход к разрешению конфликтов состоит в том, чтобы не предотвращать их заранее, а реагировать на них в момент возникновения. При возникновении конфликта между двумя агентами, один из них уступает дорогу — отступает назад до ближайшего участка, на котором встречное движение допустимо, — после чего движение продолжается в штатном режиме.

Подобная схема может быть реализована в децентрализованном виде: агенты самостоятельно договариваются о том, кому уступать, обмениваясь сообщениями, либо вовсе действуют по локальным правилам без явной коммуникации. Применимость такого подхода, однако, существенно ограничена — она требует от агентов наличия достаточно полной информации об окружении и средств синхронизации действий, что доступно далеко не всегда.

Альтернативой является централизованная реализация, в которой решения об отступлении принимаются единым управляющим контуром. Здесь возможны два варианта: автоматическое разрешение по формальному правилу и ручное вмешательство удалённого оператора. Второй вариант обладает заметным недостатком: число операторов всегда существенно меньше числа

агентов, что приводит к дополнительным задержкам при разрешении конфликта, а также сопровождается дополнительными операционными расходами. В данной работе рассматривается автоматизированный вариант.

Централизованный реактивный метод разрешения конфликтов заключается в следующем. Поскольку на широких рёбрах E_w (4) встречное движение допустимо физически, согласование требуется только для узких рёбер E_n . Каждому агенту $i \in A$ в каждый момент времени t сопоставляется одно из трёх состояний:

- движение — агент движется по своему маршруту в прямом направлении;
- отступление — агент движется в обратном направлении вдоль текущего ребра, отступая к его началу;
- ожидание — агент остановлен и ожидает разрешения на продолжение движения.

Введём параметр $\delta > 0$ — минимальный безопасный продольный зазор между двумя агентами на одном ребре, ниже которого они считаются соприкоснувшимися. Для агента i , находящегося на ребре $e = \{u, v\} \in E_n$ и движущегося из u в v , обозначим через $x_i(t) \in [0, W(e)]$ пройденное им расстояние от вершины u вдоль ребра.

Состояние системы дополнительно описывается множеством активных конфликтов $\mathcal{C}(t)$. Каждый конфликт $c \in \mathcal{C}(t)$ привязан к конкретному узкому ребру $e_c \in E_n$ и характеризуется парой направлений: «побеждающим» $u_c \rightarrow v_c$ и «проигрывающим» $v_c \rightarrow u_c$. С конфликтом ассоциированы два множества агентов:

- $P_c(t) \subseteq A$ — множество «толкающих» агентов, движущихся по e_c в направлении $u_c \rightarrow v_c$ и удерживающих конфликт активным;
- $L_c(t) \subseteq A$ — множество «проигравших» агентов, исходно двигавшихся в направлении $v_c \rightarrow u_c$ и в настоящий момент находящихся в состоянии отступления либо ожидания.

Конфликт c порождается, когда на ребре $e \in E_n$ обнаруживается пара агентов i, j , движущихся навстречу друг другу с зазором меньше δ :

$$W(e) - x_i(t) - x_j(t) < \delta. \quad (6)$$

Из пары $\{i, j\}$ проигравшим объявляется агент, прошедший по своему

ребру меньшее расстояние (то есть более удалённый от противоположного конца ребра); при равенстве выбирается произвольный агент (например, с меньшим номером). Проигравший переводится в состояние отступления и помещается в L_c , второй агент включается в P_c . Это эвристическое правило приводит к минимизации времени освобождения ребра.

Дальнейшая динамика конфликта определяется тремя сценариями.

Во-первых, агент $\ell \in L_c$ в состоянии отступления продолжает движение в обратном направлении до тех пор, пока полностью не покинет ребро e_c . После этого он переводится в состояние ожидания и остаётся в нём, не изменяя своего положения, до момента полного разрешения конфликта.

Во-вторых, конфликт может разрастаться за счёт каскадного включения других агентов. Если на ребре e_c в направлении $u_c \rightarrow v_c$ позади какого-либо $p \in P_c$ оказывается агент в состоянии движения на расстоянии меньше δ , он также включается в P_c как новый толкающий — иначе он догонит и собьёт p . Аналогично, если позади какого-либо $\ell \in L_c$ (то есть в направлении $v_c \rightarrow u_c$) на расстоянии меньше δ оказывается агент в состоянии движения, он включается в L_c и переводится в отступление, так как продолжение движения вперёд приведёт его к столкновению с отступающими.

В-третьих, конфликт c разрешается в тот момент, когда $P_c(t) = \emptyset$, то есть все толкающие агенты покинули ребро e_c . Тогда все агенты из L_c переводятся обратно в состояние движения и возобновляют движение по своим исходным маршрутам, а конфликт удаляется из $\mathcal{C}(t)$.

Упрощённый псевдокод работы описанного механизма приведён на рисунке 4. На каждом шаге сначала обновляются уже существующие конфликты — обрабатываются выход толкающих агентов с ребра, переход отступивших проигравших в ожидание и каскадное вовлечение соседей, — после чего обнаруживаются новые встречные пары на узких рёбрах.

Входные данные: Множество агентов A , множество активных конфликтов $\mathcal{C}(t)$, момент времени t

- 1 **для каждого конфликт $c \in \mathcal{C}(t)$ выполнять**
- 2 удалить из P_c агентов, покинувших ребро e_c ;
- 3 **для каждого агент $\ell \in L_c$ в состоянии отступления,**
 покинувший e_c **выполнять**
- 4 | перевести ℓ в состояние ожидания;
- 5 **если $P_c = \emptyset$ тогда**
- 6 | перевести всех агентов из L_c в состояние движения;
- 7 | удалить c из $\mathcal{C}(t)$ и перейти к следующему конфликту;
- 8 **для каждого агент $j \in A \setminus (P_c \cup L_c)$ в состоянии движения на**
 ребре e_c **выполнять**
- 9 | **если j движется в направлении $u_c \rightarrow v_c$ и приблизился к**
 некоторому $p \in P_c$ ближе чем на δ **тогда**
- 10 | добавить j в P_c ;
- 11 | **иначе если j движется в направлении $v_c \rightarrow u_c$ и приблизился**
 к некоторому $\ell \in L_c$ ближе чем на δ **тогда**
- 12 | добавить j в L_c и перевести в состояние отступления;
- 13 **для каждого узкое ребро $e \in E_n$, на котором нет активного**
 конфликта **выполнять**
- 14 **если на e существует пара встречных агентов i, j с зазором**
 меньше δ (6) **тогда**
- 15 | определить проигравшего ℓ и толкающего p по эвристике;
- 16 | создать конфликт c с $P_c = \{p\}$, $L_c = \{\ell\}$ и добавить в $\mathcal{C}(t)$;
- 17 | перевести ℓ в состояние отступления;

Рисунок 4 – Псевдокод реактивного разрешения конфликтов
управлением разъездом агентов

Описанный метод не требует какой-либо предварительной координации маршрутов и применим к любым индивидуальным планам, построенным независимо для каждого агента. От агентов требуется лишь возможность двигаться по ребру в обратном направлении и останавливаться в вершине, что

является подзадачей локального управления. Отметим, что метод не гарантирует быстрого разрешения конфликта: при стечении обстоятельств отступающий агент сам может оказаться в роли толкающего в смежном конфликте, что в плотных сценариях приводит к каскадным остановкам, но с течением времени все конфликты оказываются разрешены.

2.3.2 Предотвращение конфликтов при помощи глобальных ограничений

Другой способ исключить столкновения между агентами — запрещать агентам оказываться в конфликтных ситуациях. В тот момент, когда агент собирается войти на потенциально опасный участок среды, его проход регулируется глобальным правилом, общим для всех агентов и заранее заданным на уровне самого графа G (1). Тем самым корректной реализации правил конфликты на исполнении принципиально невозможны.

Рассматриваемый ниже метод также опирается на введённое ранее разбиение множества рёбер на широкие и узкие $E = E_w \sqcup E_n$ (4). На широких рёбрах конфликт между агентами физически невозможен и никаких дополнительных ограничений не требуется. На узких рёбрах, напротив, сосредоточены все потенциальные столкновения, и согласование движения агентов проводится именно на этих участках.

С такой точки зрения, узкие ребра можно рассматривать как общие ресурсы с ограниченным доступом, а задачу управление агентами — как задачу синхронизации разрешений на доступ к этим ресурсам. Для решения этой задачи могут быть использованы различные примитивы синхронизации, например — семафоры.

Итак, каждому узкому ребру $e = \{u, v\} \in E_n$ сопоставляется семафор — глобальный объект, регулирующий проход агентов через e . Состояние семафора в момент времени t описывается не просто счетчиком обращений, а тройкой:

$$\sigma_e(t) = (d_e(t), I_e(t), Q_e^u(t), Q_e^v(t)), \quad (7)$$

где $d_e(t) \in \{u, v, \emptyset\}$ — разрешённое направление движения по ребру (вершина, с которой агенты входят на e ; $\emptyset$ означает, что ребро свободно),

$I_e(t) \subseteq A$ — множество агентов, находящихся на ребре e ,

$Q_e^u(t)$ — очередь агентов, ожидающих прохода со стороны вершины u ,
 $Q_e^v(t)$ — очередь агентов, ожидающих прохода со стороны вершины v .

На исполнение маршрутов агентами накладывается следующее ограничение: агенту $i \in A$ разрешено начать перемещение по ребру $e \in E_n$ из вершины u в момент t тогда и только тогда, когда $d_e(t) = u$ и $i \notin Q_e^u(t) \cup Q_e^v(t)$. В противном случае агент обязан ожидать в вершине u и помещается в очередь $Q_e^u(t)$, ожидание продолжается до тех пор, пока семафор не разрешит проход в направлении $u \rightarrow v$.

Эволюция состояний семафоров во времени подчиняется двум правилам, гарантирующим отсутствие конфликтов на узких рёбрах.

Во-первых, пока на ребре находится хотя бы один агент, направление прохода не может быть изменено на противоположное:

$$I_e(t) \neq \emptyset \Rightarrow d_e(t') = d_e(t) \text{ при } I_e(t') \cap I_e(t) \neq \emptyset. \quad (8)$$

Во-вторых, как только ребро освобождается ($I_e(t) = \emptyset$), семафор выбирает новое направление прохода исходя из состояния очередей $Q_e^u(t)$ и $Q_e^v(t)$. Выбор направления может производиться по различным политикам, например, обеспечивающей минимальную задержку (первой пропускается сторона, которая дольше ожидала очереди) или минимизирующей число агентов в состоянии ожидания (первой пропускается сторона, на которой скопилось больше агентов). Все агенты, ожидающие со стороны выбранной вершины, получают разрешение на проход и переводятся в множество I_e , после чего могут одновременно двигаться по ребру в общем направлении.

Упрощённый псевдокод работы системы семафоров в некоторый момент времени приведён на рисунке 5. На каждом шаге для каждого агента, находящегося в начале узкого ребра, проверяется состояние соответствующего семафора, после чего пересматривается состояние всех семафоров и при необходимости переключается направление прохода.

Входные данные: Множество агентов A , состояния семафоров

$\{\sigma_e\}_{e \in E_n}$, момент времени t

1 для каждого агент $i \in A$, готовый войти на узкое ребро

$e = \{u, v\} \in E_n$ из вершины u **выполнять**

2 **если** $d_e(t) = u$ **тогда**

3 добавить i в $I_e(t)$ и разрешить перемещение;

4 **иначе**

5 добавить i в очередь $Q_e^u(t)$ и перевести в состояние ожидания;

6 для каждого агент $i \in A$, завершивший проход по ребру $e \in E_n$

выполнять

7 удалить i из $I_e(t)$;

8 для каждого узкое ребро $e \in E_n$ с $I_e(t) = \emptyset$ **выполнять**

9 выбрать новое направление $d_e(t)$ по политике обслуживания
очереди $Q_e^u(t), Q_e^v(t)$;

10 перевести всех агентов из очереди со стороны $d_e(t)$ в $I_e(t)$ и
разрешить им движение;

Рисунок 5 – Псевдокод управления агентами при помощи системы семафоров

Из приведённых правил непосредственно следует, что в любой момент времени все агенты, находящиеся на одном узком ребре e , движутся в одном направлении, а на широких рёбрах ограничения вообще не накладываются. Таким образом, конфликты обмена и встречные конфликты на ребре исключаются по построению, от агента требуется лишь обладать возможностью остановиться и ожидать, что уже является подзадачей локального управления.

2.4 Другие методы управления

2.4.1 Адаптивное планирование с учётом наблюдаемой загруженности среды

Описанные ранее методы разрешения конфликтов целенаправленно отказываются от многоагентного планирования и опираются на независимое

построение индивидуальных маршрутов. В простейшем случае такой маршрут строится без учёта какой-либо информации о других агентах — по статической стоимости рёбер. Однако централизованный характер управления позволяет накапливать наблюдения о том, как агенты фактически проходят граф, и использовать эти наблюдения для коррекции стоимости рёбер при последующих запусках поиска.

В качестве базового алгоритма построения индивидуального маршрута используется классический A^* [14] — информированный поиск кратчайшего пути на графе, обходящий вершины в порядке возрастания величины $f(v) = g(v) + h(v)$, где $g(v)$ — стоимость пути из стартовой вершины, а $h(v)$ — эвристическая оценка стоимости оставшегося пути до цели. При допустимой и согласованной эвристике A^* возвращает оптимальный маршрут и обходит минимально возможное число вершин среди всех алгоритмов, использующих ту же эвристику. На неориентированном планарном графе с геометрически расположенными вершинами естественной допустимой эвристикой является евклидово расстояние до цели — именно она используется при поиске по статическим весам $W(e)$ (1).

В чистом виде A^* строит маршрут исходя только из геометрии графа и не учитывает текущего состояния среды. Если в некоторой части графа сосредоточена значительная доля агентов, локально оптимальные маршруты будут продолжать проводиться через те же «короткие» рёбра, на которых уже образовался затор. В результате такие маршруты оказываются хуже формально более длинных альтернатив, проходящих через свободные участки графа: фактическое время движения по затору определяется не длиной ребра, а скоростью, с которой агенты проходят его в условиях взаимного блокирования.

Эту обратную связь можно встроить в алгоритм поиска, если оценивать стоимость ребра не его длиной, а ожидаемым временем прохождения, наблюдаемым в ходе работы системы. Идея об использовании наблюдаемых задержек на рёбрах вместо априорных весов восходит к постановке онлайн-задачи о кратчайшем пути [15]: там изучается задача поиска кратчайшего пути в условиях, когда веса рёбер произвольно меняются со временем, и для неё построены алгоритмы с доказуемой оценкой отклонения суммарной потери. В описываемой ниже схеме используется существенно более простой, статистический вариант той же идеи: оценки скоростей рёбер

строятся непосредственно по эмпирической истории прохождений и не сопровождаются формальными гарантиями оптимальности. Для этого вместе с графом ведётся набор статистик: каждому ребру $e \in E$ сопоставляется история наблюдений вида:

$$\mathcal{H}_e(t) = \{ (t_{\text{exit}}^p, v_p) \}_p, \quad (9)$$

где t_{exit}^p — момент, в который очередной агент покинул ребро e ,
 $v_p = W(e)/(t_{\text{exit}}^p - t_{\text{enter}}^p)$ — средняя скорость этого прохождения.

Каждый раз, когда какой-либо агент завершает движение по ребру, в $\mathcal{H}_e(t)$ добавляется новое наблюдение. При этом чрезмерно старые наблюдения удаляются, чтобы оценка не сохраняла память о давно прошедших состояниях среды. По истории $\mathcal{H}_e(t)$ строится оценка эффективной скорости прохождения ребра $\bar{v}_e(t)$. При её построении необходимо учитывать два обстоятельства.

Во-первых, при планировании по времени интерес представляет математическое ожидание времени прохождения ребра $\mathbb{E}[W(e)/v]$, а не величина $W(e)/\mathbb{E}[v]$. Эти величины не совпадают, и при бимодальном распределении скоростей (свободное движение и движение в заторе) арифметическое среднее систематически завышает эффективную скорость. Поэтому в качестве оценки используется взвешенное гармоническое среднее наблюждённых скоростей.

Во-вторых, более свежие наблюдения отражают актуальное состояние среды точнее, чем устаревшие. Веса наблюдений задаются экспоненциально убывающей функцией возраста $a_p = t - t_{\text{exit}}^p$:

$$w(a) = \exp(-a/\tau), \quad (10)$$

где $\tau > 0$ — характерное время забывания.

На рёбрах с редким движением это обеспечивает плавное затухание устаревших образцов вместо их скачкообразного вытеснения новыми.

Для рёбер, по которым ещё не накоплено наблюдений, оценка должна быть определена и не должна выделять такие рёбра как непомерно дорогие или дешёвые. Для этого вводится байесовский априор: фиктивное наблюдение скорости свободного движения v_0 с весом $w_0 \geq 0$, измеряемым в

условных «виртуальных проходах». Итоговая оценка эффективной скорости ребра e в момент времени t имеет вид:

$$\bar{v}_e(t) = \frac{w_0 + \sum_{p \in \mathcal{H}_e(t)} w(a_p)}{\frac{w_0}{v_0} + \sum_{p \in \mathcal{H}_e(t)} \frac{w(a_p)}{v_p}}. \quad (11)$$

Полученная оценка (11) используется в качестве стоимости ребра при поиске A^* : вес ребра e интерпретируется не как его длина, а как ожидаемое время прохождения

$$c_e(t) = \frac{W(e)}{\bar{v}_e(t)}. \quad (12)$$

Соответственно, поиск минимизирует не суммарную длину маршрута, а суммарное ожидаемое время движения по нему, что и является непосредственной целью с точки зрения качества обслуживания заданий.

При такой замене стоимости рёбер эвристика также должна выражаться в единицах времени, иначе сравнение $g(v)$ и $h(v)$ теряет смысл. В качестве эвристики используется:

$$h(v) = \frac{\|v - g_i\|_2}{v_{\max}}, \quad (13)$$

где $v_{\max}$ — максимально достижимая агентами скорость.

Такая эвристика остаётся допустимой и согласованной: евклидово расстояние не превышает длины любого пути в графе, а делитель $v_{\max}$ гарантирует, что оценка времени никогда не превосходит фактического времени движения по любому маршруту, в том числе и в условиях свободной среды ($\bar{v}_e(t) \leq v_{\max}$). Следовательно, корректность и оптимальность A^* по выбранной целевой функции сохраняются.

Описанный метод сочетается с любым из рассмотренных в предыдущем разделе способов разрешения конфликтов: коррекция весов рёбер происходит на этапе планирования и не накладывает на исполнение никаких дополнительных требований сверх тех, что уже предъявляются выбранным методом разрешения конфликтов. С другой стороны, замедления, вызванные работой механизма разрешения конфликтов автоматически отражаются в

наблюдаемых скоростях v_p и через них — в стоимости рёбер при последующих запусках поиска. Тем самым планировщик постепенно перенаправляет агентов вокруг систематически перегруженных участков графа, что в благоприятных сценариях способно улучшить интегральные показатели исполнения маршрутов.

2.5 Метрики

Для сравнения различных методов централизованного управления автономными агентами требуется ввести количественные показатели качества получаемых решений.

В классической постановке задачи MAPF решением является набор индивидуальных планов $\Pi = \{\pi_1, \dots, \pi_k\}$. Обозначим момент, в который агент i впервые оказывается в своей целевой вершине g_i , как $|\pi_i|$. В качестве показателей качества такого решения в литературе [5] наиболее часто используются следующие метрики.

Общее время выполнения (makespan) определяется как момент, к которому все агенты достигают своих целей, и характеризует общее время исполнения решения:

$$MS(\Pi) = \max_{1 \leq i \leq k} |\pi_i|. \quad (14)$$

Сумма стоимостей (sum of costs, flowtime) определяется как суммарное время, затраченное всеми агентами на достижение целей:

$$SOC(\Pi) = \sum_{i=1}^k |\pi_i|. \quad (15)$$

В отличие от MS, SOC поощряет решения, в которых агенты в среднем достигают целей раньше, и наиболее распространена в работах по поисковым алгоритмам MAPF, в том числе в описанных выше CBS и CCBS. Близка по смыслу метрика суммы перемещений (sum of fuel), в которой учитываются только действия перемещения, но не действия ожидания — она отражает суммарные энергетические затраты на выполнение плана. Наконец, в постановках с жёстким ограничением на время выполнения плана в качестве показателя качества используется число агентов, успевших достичь своей

цели до заданного момента T , то есть $|\{i \in A : |\pi_i| \leq T\}|$.

Помимо качества самого плана, обычно фиксируются и характеристики работы алгоритма — время планирования, число раскрытых узлов в дереве поиска, доля задач, решённых в пределах заданного бюджета времени (success rate). Эти показатели важны при сравнении вычислительной эффективности методов, но не отражают качество поведения агентов в среде.

Все перечисленные метрики разработаны для «одноразовой» постановки MAPF, в которой для каждого агента задана единственная целевая вершина, а после выполнения построенного решения задача считается завершённой. В рассматриваемой постановке такое допущение неприменимо: как уже отмечалось, после достижения очередной цели агенту назначается новое целевое местоположение, и однозначно выделенного момента окончания плана у агента i нет. Поэтому величина $|\pi_i|$ в классическом смысле не определена, а значения MS (14) и SOC (15) неограниченно растут с увеличением горизонта наблюдения и не позволяют сопоставлять методы между собой.

Кроме того, при разрешении конфликтов путём перепланирования план π_i может многократно изменяться в процессе работы системы, а момент достижения очередной цели зависит не только от исходных данных, но и от истории взаимодействия с другими агентами. В таких условиях метрики, опирающиеся на конкретный итоговый набор маршрутов, теряют смысл стационарного показателя: их значения определяются моментом, в который остановлено наблюдение, а не свойствами самого метода. Естественная для задач разрешения конфликтов метрика — число возникающих столкновений — также неприменима, поскольку практически все рассматриваемые методы не допускают конфликтов между агентами.

Поэтому в качестве показателей качества в рассматриваемой задаче целесообразно использовать интегральные характеристики потока заданий на фиксированном промежутке времени: число достигнутых системой целей, среднее время обслуживания одного задания и среднюю скорость движения агента по маршруту. Такие показатели прямо обобщают SOC и число завершённых задач на непрерывный режим работы.

3 СИМУЛЯЦИЯ ПОВЕДЕНИЯ АВТОНОМНЫХ АГЕНТОВ В ГОРОДСКОЙ СРЕДЕ

Сравнительная оценка методов глобального планирования требует воспроизводимой среды, в которой можно одинаковым образом запускать различные алгоритмы на одних и тех же входных данных и измерять выбранные показатели качества. Натурный эксперимент с реальными доставочными агентами для этой цели плохо приспособлен: он дорог, плохо повторяем и не позволяет в разумные сроки перебрать достаточное количество конфигураций дорожной сети и плотности агентов. Поэтому в настоящей работе предложен и реализован симулятор, выступающий основным инструментом экспериментального исследования.

Симулятор должен решать следующие задачи:

- воспроизводить движение множества агентов по графу дорожной сети;
- предоставлять единый интерфейс для подключения как методов планирования, заранее учитывающих возможные конфликты, так и методов непосредственного разрешения конфликтов;
- обеспечивать загрузку графов, построенных по реальным фрагментам городской среды, и генерацию конфигураций агентов с заданными параметрами;
- вычислять метрики качества и сохранять результаты прогонов в виде, пригодном для последующего статистического сравнения.

3.1 Архитектура симулятора

Симулятор построен как набор связанных модулей, каждый из которых отвечает за один аспект моделируемой системы: представление дорожной сети, поведение отдельного агента, выдачу заказов, глобальное планирование маршрутов, локальное разрешение конфликтов на узких участках, накопление показателей качества и визуализацию. Такое разделение позволяет, фиксируя все остальные элементы, заменять отдельные компоненты — в первую очередь планировщик маршрутов — и сравнивать различные алгоритмы планирования в одинаковых условиях. Общая схема взаимодействия модулей в рамках одного шага моделирования приведена на рисунке 6. Сплошные стрелки соответствуют передаче состояния и управления в ходе одного шага

моделирования, пунктирные — зависимостям по данным. Пунктирным контуром выделены необязательные или заменяемые модули.

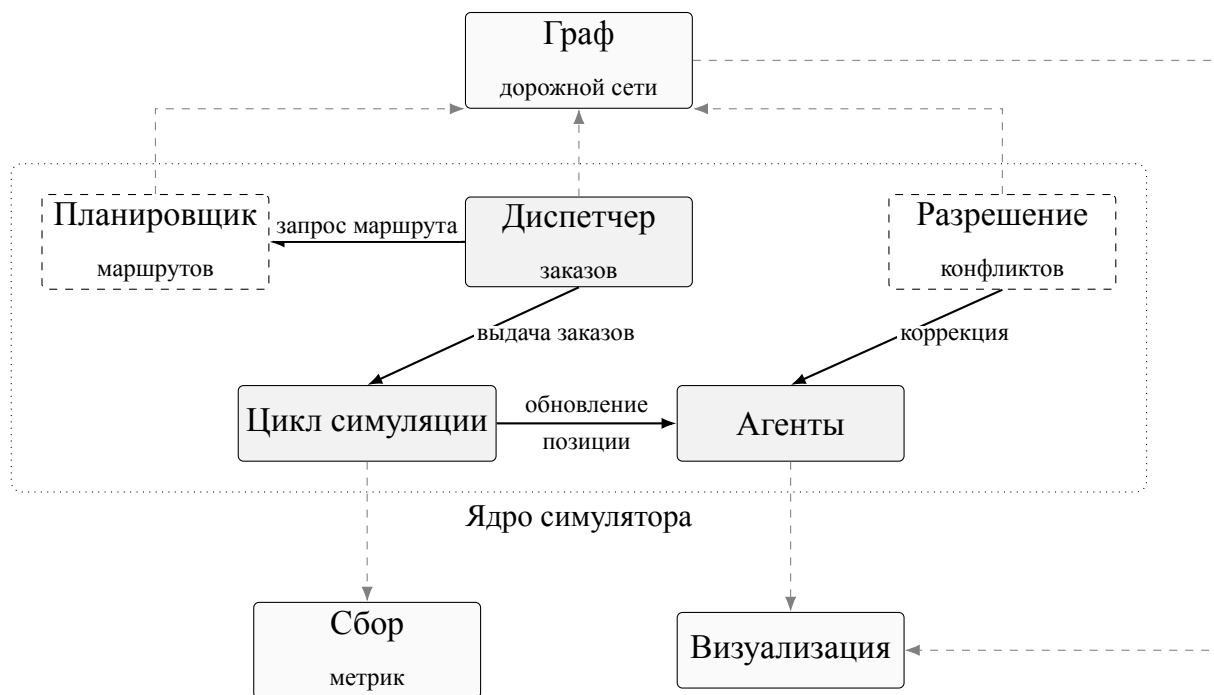


Рисунок 6 – Структура симулятора и взаимодействие основных компонент.

Граф дорожной сети — центральная структура входных данных, описывающая статическую часть моделируемой среды. Содержит множество вершин с указанием их типа и множество неориентированных взвешенных рёбер, для каждого из которых дополнительно задан признак «узкое место» — участок, на котором встречное движение запрещено и потому возможны конфликты между агентами. Граф является общим неизменяемым источником данных для всех остальных компонент системы.

Агент представляет из себя модель отдельного автономного исполнителя заказов. В ней описано его текущее положение в пространстве, состояние и привязанный к агенту маршрут, вдоль которого он перемещается. Агент отвечает за собственную пространственно-временную динамику: продвижение по маршруту на заданный временной интервал с учётом возможного расписания прохождения вершин и ограничений скорости, накладываемых характером участка дорожной сети.

Диспетчер заказов является источником целей и маршрутов для агентов. На каждом шаге симуляции он выявляет свободных агентов, назначает им новые точки доставки из множества допустимых, запрашивает у

планировщика маршрут от текущего положения агента к назначенной точке. После завершения заказа диспетчер отправляет агента на ближайшую базовую точку.

Планировщик маршрутов — это подключаемая компонента, отвечающий за глобальное планирование. По запросу диспетчера строит маршрут от заданной начальной вершины графа к заданной конечной. Планировщик предоставляется через единый интерфейс, что позволяет на одном и том же графе и одной и той же конфигурации агентов сравнивать разные алгоритмы планирования.

Разрешение конфликтов осуществляется другой подключаемой компонентой локального уровня, активируемой в тех случаях, когда выбранный планировщик самостоятельно не гарантирует отсутствия конфликтов агентов. По текущему состоянию агентов и графа выявляются возникающие на узких участках конфликты и корректируется поведение конфликтующих агентов, тем самым обеспечивается выполнение пропускных ограничений узких мест.

Цикл симуляции — управляющее ядро системы. Реализует дискретно-временной цикл: на каждом шаге фиксированной длительности поочередно вызывает диспетчера, продвигает агентов вдоль их маршрутов, передаёт управление разрешителю конфликтов и, при необходимости, визуализатору. Цикл симуляции связывает остальные компоненты и определяет момент завершения симуляции.

Подсистема статистики отвечает за измерение качества работы системы в ходе симуляции. Аккумулирует данные, поступающие от различных источников: от агентов — фактическую скорость и время прохождения рёбер, от диспетчера — время выполнения заказов, от разрешителя конфликтов — факты и длительности задержек на узких участках, от планировщика — характеристики строящихся маршрутов.

Визуализатор — это подсистема качественного анализа. По текущему состоянию агентов и геометрии графа формирует растровое изображение моделируемой среды в некоторые моменты времени.

3.2 Компоненты симулятора

3.2.1 Цикл симуляции

Цикл симуляции реализует дискретно-временную модель: вся работа симулятора сводится к продвижению виртуального времени t от нуля до заданной длительности T и выполнению на каждом значении t некоторого фиксированного набора действий. Цикл также отвечает за определение момента завершения прогона: симуляция останавливается, как только t превышает T .

Внутри цикла используются три независимых периодических процесса с собственными шагами:

- основной шаг симуляции длительностью Δt , на котором происходит продвижение состояния системы и который определяет точность дискретизации непрерывного движения агентов;
- шаг визуализации $\Delta t_{\text{vis}} \geq \Delta t$, на котором текущее состояние сохраняется как кадр будущей видеозаписи; визуализация может быть полностью отключена, и тогда соответствующий процесс не запускается;
- шаг журналирования прогресса, используемый для информирования о ходе длительных прогонов и не влияющий на состояние модели.

Эти процессы объединяются в общем расписании событий, упорядоченном по виртуальному времени, а при совпадении моментов — по фиксированным приоритетам. На каждой итерации цикл извлекает ближайшее по времени событие, исполняет соответствующее действие и возвращает событие в расписание со сдвигом на его период. Такая организация обеспечивает корректное чередование разнородных процессов, и при необходимости позволяет добавлять новые периодические задачи без изменения структуры цикла.

Содержательная часть основного шага представляет собой строго упорядоченную последовательность фаз, выполняемых на каждом шаге Δt :

- а) локальное разрешение конфликтов,
- б) распределение заказов,
- в) продвижение агентов.

В таком виде цикл симуляции остаётся одинаковым для всех

конфигураций сравниваемых алгоритмов. Это обеспечивает требуемую от экспериментальной среды воспроизводимость.

3.2.2 Продвижение агентов

Агент моделирует отдельного автономного исполнителя заказов. Он описывается текущим положением на плоскости, дискретным состоянием и привязанным к нему маршрутом — структурой, отвечающей за пространственно-временную динамику движения вдоль ломаной, выданной планировщиком.

Положение агента на маршруте задаётся скалярной координатой $x \in [0, L]$, где L — полная длина ломаной, а текущая точка в плоскости получается интерполяцией маршрута по x . Помимо этого хранится «остаточный» маршрут — ломаная, начинающаяся в текущей позиции и продолжающаяся до конца исходной, — именно она используется подсистемой разрешения конфликтов.

Множество состояний агента состоит из четырёх элементов: *простой*, *движение*, *ожидание* и *отступление*. Простой соответствует агенту без активного маршрута либо агенту, только что завершившему текущий заказ. В состоянии движения агент за шаг Δt продвигается вперёд на величину $v \Delta t$, которое может быть ограничено сверху (подробнее в разделе 3.3). Состояние ожидания означает, что агент остановлен внешним механизмом, в таком состоянии агент не двигается. Состояние отступления также используется для разрешения конфликтов: агент движется в обратном направлении вдоль уже пройденной части маршрута с пониженной скоростью.

Переходы между состояниями инициируются исключительно внешними по отношению к агенту компонентами — диспетчером заказов и разрешителем конфликтов. Сам агент лишь самостоятельно фиксирует окончание маршрута, переводя себя из движения в простой при $x = L$. Полная схема переходов приведена на рисунке 7.

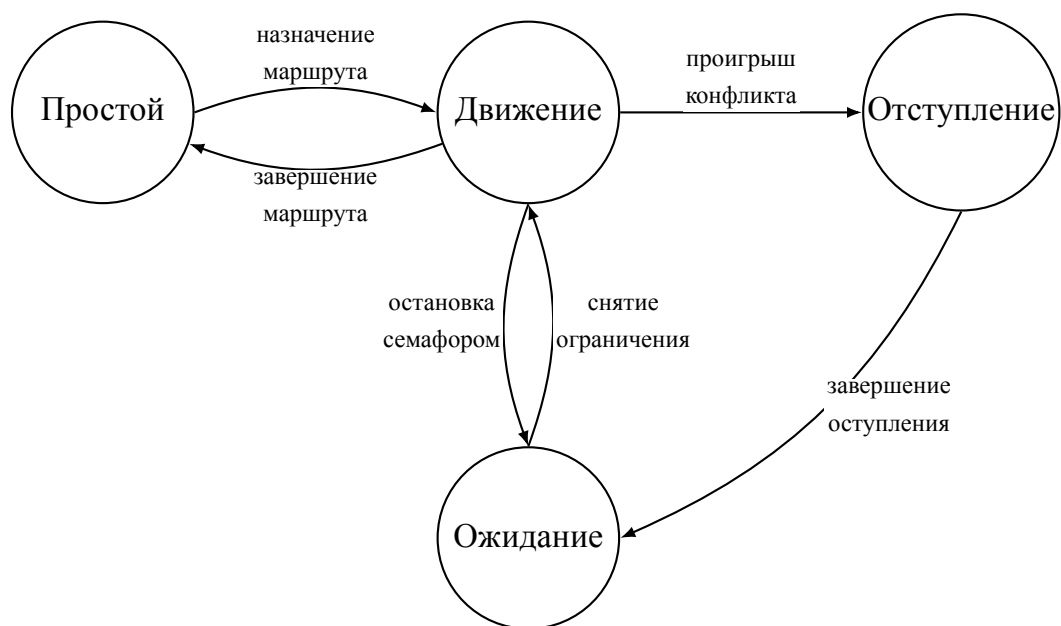


Рисунок 7 – Диаграмма состояний агента и переходов между ними.

Содержательная динамика реализуется в фазе продвижения агентов основного шага цикла симуляции. Для каждого активного агента вычисляется его эффективная скорость с учётом типа текущего сегмента маршрута (обычный или узкий) и направления движения относительно исходной ориентации ребра. После этого, в зависимости от состояния, выполняется либо продвижение вперёд на $v \Delta t$, либо обратный ход с понижающим коэффициентом. Точные значения скоростных коэффициентов приведены в разделе 3.3.

Со стороны агента происходит также учёт прохождения рёбер для подсистемы статистики (подробнее про подсистему статистики в разделе 3.2.6). В маршруте хранится для каждого сегмента время первого входа агента в прямом направлении и индекс максимального достигнутого сегмента. В момент пересечения границы между двумя сегментами вычисляется точное время выхода линейной интерполяцией по фактическому смещению за шаг, и в накопитель статистики передаётся запись о пройденном ребре с временами входа и выхода. Если в дальнейшем агент откатывается и повторно проходит тот же сегмент вперёд, повторная запись не создаётся: в статистику попадает единый интервал от первоначального входа до окончательного выхода.

3.2.3 Разрешение конфликтов

Разрешение конфликтов оформлено как подключаемый локальный компонент. Это позволяет в рамках одного эксперимента подменять конкретную стратегию, не затрагивая остальные модули. Реализованы две стратегии разрешения, а также есть возможность вовсе не использовать разрешение конфликтов.

В цикле симуляции разрешение конфликтов вызывается первым на каждом основном шаге. Такой порядок гарантирует, что любое изменение состояния агента будет учтено уже на текущем шаге, и продвижение конфликтующих агентов выполнится с поправкой на новое состояние.

При разрешении конфликтов воздействие на остальную систему ограничивается изменением дискретного состояния агентов: компонента переводит выбранных агентов между состояниями *движения*, *ожидания* и *отступления*, не вмешиваясь ни в их положение, ни в назначенные маршруты.

3.2.3.1 Разрешение конфликтов при помощи семафоров

При инициализации этой стратегии производится однократный обход графа: для каждого узкого ребра создаётся отдельный объект-семафор с двумя приоритетными очередями ожидания и двумя множествами агентов, уже находящихся внутри ребра и движущихся в соответствующих направлениях. Все семафоры дополнительно загружаются в пространственный индекс по описывающим прямоугольникам своих участков, что позволяет на каждом шаге для очередного агента быстро находить ограниченное число семафоров, потенциально расположенных на его ближайшем сегменте маршрута, а не выполнять полный перебор всех узких рёбер графа.

Принятие решения о постановке в очередь привязано к малому пороговому расстоянию между текущим положением агента и началом узкого ребра. Этот порог введён для того, чтобы агент не блокировался слишком рано, но и не пропускался уже после физического въезда на ребро.

В ходе работы эта компонента также накапливает информацию о времени ожидания для каждого из агентов и передает ее в подсистему статистики.

3.2.3.2 Разрешение конфликтов управлением разъезда

Этот вариант разрешения конфликтов не требует предварительного

обхода графа и хранит лишь те конфликты, что фактически возникли в ходе симуляции. Каждый активный конфликт представлен записью, содержащей неориентированный ключ ребра, направление прохода победителей, список толкающих агентов, два множества проигравших (отдельно для отступающих и для уже остановившихся в ожидании), а также таблицы времён начала ожидания и отступления, ведущиеся отдельно по каждому вовлечённому агенту.

При обнаружении конфликта в роли проигравшего детерминировано выбирается тот агент, который меньше продвинулся по ребру. При точном совпадении смещений выбирается агент с меньшим идентификатором.

Эта компонента собирает информацию о времени нахождения агентов в состояниях ожидания и отступления, а также о количестве конфликтных встреч агентов и передает ее в подсистему статистики.

3.2.4 Распределение заказов

Диспетчер заказов служит источником целей и маршрутов для агентов. При инициализации он извлекает из графа два особых множества вершин — базовые точки и точки доставки — и циклически распределяет базы между агентами: i -му агенту приписывается база с номером $i \bmod B$, где B — общее число баз, и его стартовое положение совмещается с координатами этой вершины. Так достигается равномерное распределение агентов по базовым точкам.

На каждом основном шаге для каждого агента независимо от его состояния обновляется накопленная пройденная длина: к ней прибавляется евклидово расстояние между позицией на предыдущем шаге и текущей позицией. Такой учёт намеренно опирается на фактические смещения, а не на длину запланированного маршрута, что делает метрику устойчивой к переназначениям маршрута со стороны глобального планировщика, к ожиданиям и отступлениям.

Содержательное действие диспетчера распространяется лишь на агентов в состоянии простоя. Логика выбора нового назначения построена на двухфазном цикле «базовая точка — точка доставки — базовая точка»: если активного заказа у агента нет, диспетчер равномерно случайно выбирает точку доставки и запрашивает у планировщика маршрут от базы к ней, иначе запрашивается обратный маршрут.

Диспетчер заказов является основным источником сведений о производительности доставки: суммарного числа выполненных заказов, времени выполнения заказа и средней скорости агентов, — именно эти показатели используются как основные метрики для сравнения различных стратегий.

3.2.5 Построение глобальных маршрутов

Планировщик маршрутов реализован как подключаемая компонента с единым интерфейсом: по запросу, содержащему пару вершин графа, текущее виртуальное время и идентификатор обратившегося агента, он возвращает геометрию маршрута и сопровождающую её последовательность вершин. Все рассматриваемые алгоритмы реализуют этот интерфейс. Помимо самой процедуры поиска, интерфейс выделяет две настраиваемые величины: эвристическую оценку расстояния между парой вершин и стоимость прохода ребра в заданный момент времени.

3.2.5.1 A^* и адаптивный A^*

Классический A^* реализован стандартным образом [14]: эвристикой служит евклидово расстояние между вершинами, стоимостью прохода ребра — его длина, поиск ведётся с очередью с приоритетом по сумме накопленной стоимости и эвристики.

Адаптивный A^* отличается тем, что использует не статические длины рёбер, а оценки фактической скорости их прохождения, накапливаемые подсистемой статистики (см. раздел 3.2.6). Эвристика в этом случае измеряется в единицах времени и равна евклидовому расстоянию, делённому на максимально допустимую скорость движения, такая оценка остаётся допустимой при любом наполнении статистики. Стоимость прохода ребра вычисляется как длина ребра, делённая на оценку средней скорости; при отсутствии накопленной информации используется максимально допустимая скорость, что согласует поведение алгоритма со статическим A^* в начале симуляции.

Запрос средней скорости на ребре — сравнительно дорогая операция: помимо просмотра таблицы наблюдений она требует отбрасывания устаревших записей и взвешенного гармонического усреднения с

экспоненциальным затуханием. Поскольку в пределах одного шага симуляции момент времени для всех вызовов одинаков, а сами наблюдения за шаг не обновляются, в адаптивном A^* предусмотрено покадровое кэширование оценок скорости по неориентированному ключу ребра. Кэш инвалидируется при переходе к следующему моменту времени, или при любом другом изменении накопленных данных.

3.2.5.2 CCBS

Реализация CCBS опирается на эталонный код алгоритма [16], выступающий в роли самостоятельной библиотеки, и оборачивается переходным слоем, согласующим её внутреннее представление графа и маршрутов с принятым в остальной системе. По сравнению с оригинальной версией добавлены адресация конкретного агента-инициатора запроса, ограничение времени работы решателя в виде двух последовательных бюджетов и отдельная процедура одноагентного поиска против заранее зафиксированных расписаний остальных агентов.

В отличие от одноагентных планировщиков, обращение к CCBS неизбежно затрагивает не только требуемого агента, но и тех агентов, чьё движение пересекается с его потенциальным маршрутом. Прямолинейное перепланирование всех агентов оказалось бы неприемлемо дорогим, поэтому решение строится в виде трёхуровневой последовательности попыток.

На первом уровне выполняется одноагентный поиск для требуемого агента против зафиксированных расписаний остальных агентов. Маршрут строится при помощи алгоритма SIPP, который является частью библиотеки CCBS. При успехе маршрут немедленно фиксируется, а расписания других агентов остаются нетронутыми.

Если одноагентный поиск не дал решения, выполняется переход на второй уровень — совместное многоагентное планирование. Чтобы не передавать в решатель заведомо нерелевантных агентов, вначале строится грубая оценка маршрута требуемого агента в виде одноагентного поиска без ограничений, после чего в совместную задачу включаются только те агенты, последовательности вершин которых имеют хотя бы одну общую вершину с этим маршрутом. Сам совместный поиск запускается с двухуровневым ограничением по времени: сначала с коротким бюджетом, и лишь при истечении этого бюджета (но не при формальной неразрешимости) — с более

длинным. Такая схема даёт быстрый ответ в подавляющем большинстве простых случаев и одновременно сохраняет полноту на трудных конфигурациях.

Если и совместный поиск не привёл к решению, выполняется ещё одна попытка одноагентного поиска против обновлённых расписаний соседей, а в случае её неудачи — безусловный откат к классическому A^* .

Поскольку CCBS уже самостоятельно гарантирует отсутствие встречных проходов на узких рёбрах в рамках возвращаемого расписания, локальное разрешение конфликтов при его использовании не запускается. Согласование между планировщиком и фазой продвижения агента в этом случае обеспечивается следующим образом: к маршруту дополнительно прикрепляется неубывающая последовательность плановых времён прибытия в вершины. Совпадающие подряд идентификаторы вершин со строго возрастающими плановыми временами кодируют явную остановку в этой вершине. Расписание используется как верхняя граница: при продвижении вперёд x дополнительно ограничивается линейной интерполяцией расписания в текущий момент времени, что естественным образом порождает паузы, продиктованные глобальным планировщиком.

3.2.6 Сбор статистик

Подсистема сбора статистик отделена от прикладных модулей и оформлена как единое хранилище, к которому имеют доступ все участники симуляции: цикл симуляции, агенты, диспетчер заказов, разрешитель конфликтов и планировщики маршрутов. Такая централизация позволяет каждому модулю фиксировать наблюдаемые им события в точке их возникновения, не зная ни о существовании остальных накопителей, ни о способе последующей агрегации, и тем самым избавляет основной цикл от служебной обвязки.

Хранилище объединяет накопители двух типов. Первый — скалярные счётчики дискретных событий, увеличиваемые на единицу при наступлении соответствующего происшествия. Второй — кумулятивные накопители величин, сохраняющие весь набор зарегистрированных наблюдений и предоставляющие по запросу как агрегированные характеристики (сумма, выборочное среднее), так и исходную выборку, пригодную для последующего статистического анализа за пределами симулятора.

Состав итогового отчёта зависит от того, какие модули активны в данном прогоне. Для этого введён реестр метрик: каждая метрика регистрируется один раз вместе с описанием, ссылкой на соответствующий накопитель и необязательным предикатом, проверяющим контекст запуска (выбранный планировщик, выбранный разрешитель и т. п.).

В подсистеме сбора статистик также реализован накопитель статистик по рёбрам графа, обслуживающий планировщик, опирающийся на эмпирические оценки времени проезда. При каждом пересечении агентом очередной вершины маршрута фиксируется ребро, по которому совершён переход, момент входа, момент выхода и пройденная длина. По этим данным накопитель вычисляет наблюденную путевую скорость и помещает её в историю соответствующего ребра. История поддерживается с помощью двух согласованных механизмов забывания — жёсткой границы по возрасту, при достижении которой наблюдения исключаются из памяти, и плавного экспоненциального ослабления вклада старых наблюдений. Кроме того, история может быть дополнена априорными наблюдениями с максимальной скоростью. Запрос текущей оценки скорости для ребра сводит накопленную историю к взвешенному среднему по формулам, приведённым в разделе 2.4.1. Ребро адресуется неупорядоченной парой вершин, что отражает симметричный характер наблюдений по полосам обоих направлений.

3.2.7 Пользовательский интерфейс

Симулятор оформлен как консольное приложение, принимающее параметры запуска через аргументы командной строки. Обязательным параметром является путь к файлу с описанием графа дорожной сети, который может быть задан как в собственном текстовом формате, так и в формате GeoJSON [17].

Множество флагов разбивается на несколько смысловых групп. Первая задаёт параметры самой симуляции: длительность симуляции, шаг дискретизации, число агентов, их максимальную скорость и ограничение на количество базовых точек, извлекаемых из карты. Вторая выбирает используемые алгоритмы — планировщик маршрутов и метод разрешения конфликтов на узких рёбрах. Третья группа управляет адаптивным планировщиком: размером окна наблюдений, константой экспоненциального затухания и весом априорного распределения. Четвёртая отвечает за

визуализацию: каталог для сохранения кадров, период их генерации, размер кадра, дополнительный масштаб объектов, а также набор флагов, выборочно включающих отрисовку графа, агентов, точек, узких рёбер и тепловой карты статистик. При указании выходного видеофайла сгенерированные кадры автоматически склеиваются в анимацию средствами `ffmpeg` [18].

По завершении симуляции выводится итоговый отчёт по метрикам, состав которого определяется реестром, описанным в разделе 3.2.6, с учётом фактически выбранных в данном запуске компонент.

3.3 Ограничения и допущения

Несмотря на то, что целевой задачей симуляции является воспроизведения городской среды, в симулятор включён лишь ограниченный набор явлений. Прочие аспекты дорожного движения сознательно вынесены за рамки модели. Ниже приведены принятые упрощения и численные значения параметров.

Исходными данными для построения графа дорожной сети служат фрагменты реальной городской карты, которые перед использованием подвергаются значительной обработке и упрощению. Подробное описание процедуры подготовки данных приведено в разделе 3.4.

Все агенты считаются однотипными и передвигаются с одинаковой максимальной скоростью $v_{\max} = 2.2$ м/с. Для имитации несовершенства локального планирования вводятся понижающие коэффициенты: на рёбрах, помеченных как узкие, скорость ограничена величиной $0.75 \cdot v_{\max}$, а при движении в обратном направлении — величиной $0.5 \cdot v_{\max}$.

Агенты моделируются материальными точками, однако на одновременное нахождение нескольких сонаправленных агентов в одной точке наложены ограничения, отражающие физический размер агентов и окружения: не более трёх агентов на обычном ребре и не более одного — на узком. При нарушении этого условия идущий следом агент замедляется до тех пор, пока расстояние до следующего кластера не превысит 0.8 м. Ограничения не применяются к агентам, находящимся в базовых точках и точках доставки, поскольку соответствующие вершины интерпретируются как площадки, на которых физическое пересечение агентов несущественно.

Симулируется поведение исключительно автономных агентов: не учитывается влияние прочих участников дорожного движения (автомобилей,

пешеходов, велосипедистов и т. п.), работа регулируемых и нерегулируемых пешеходных переходов, а также внештатные ситуации вроде временного перекрытия проезда.

Единицей глобального перемещения агента является заказ — последовательный проезд от базовой точки до точки доставки и обратно. Имитируется режим максимальной загруженности системы доставки: по достижении точки доставки агенту немедленно назначается возвращение в его базовую точку, а по прибытии на базовую точку — новый заказ. Конечная точка очередного заказа выбирается равновероятно из общего пула точек доставки, что приводит к увеличению средней длины маршрута при увеличении размера графа.

3.4 Получение исходных данных

Для построения графа дорожной сети используются открытые данные проекта OpenStreetMap (OSM) [19]. Произвольный фрагмент карты, выгруженный в формате .osm, подвергается многоступенчатой обработке, реализованной в отдельной утилите. Цель обработки — получить компактное планарное представление пешеходной сети с явно выделенными базовыми точками и точками доставки, удовлетворяющее формальным требованиям симулятора: каждое ребро представлено отрезком из двух вершин, граф состоит из одной компоненты связности, рёбра не пересекаются во внутренних точках.

Входными данными процедуры служат исходный OSM-файл и опциональный GeoJSON-полигон, очерчивающий интересующую территорию (если файл не указан, выгрузка для ограничивающего прямоугольника полигона автоматически запрашивается через Overpass API).

Из исходного OSM-файла отбираются объекты следующих типов:

- пешеходные пути — линейные объекты (way) с тегом highway из перечня footway, pedestrian, path, crossing, living_street, track, а также прочие пути, у которых тег foot разрешает пешеходное движение (yes, designated, permissive);
- базовые точки — точечные объекты (node), помеченные как дарксторы (dark_store=yes) или супермаркеты (shop=supermarket); недействующие, строящиеся и

заброшенные объекты отбрасываются;

- точки доставки — точки со значением `entrance` из перечня `yes, main, staircase, home`, соответствующих входам в жилые здания;
- препятствия — замкнутые контуры зданий (`building`) и открытые ломаные ограждений (`barrier` со значениями `wall, fence, kerb`) — используются исключительно как вспомогательные геометрические объекты на последующих этапах.

Дальнейшая обработка выполняется последовательностью шагов, каждый из которых приводит данные ближе к требуемому формату:

- а) Пространственная фильтрация: если задан полигон территории, все объекты, хотя бы одна вершина которых лежит вне полигона, отбрасываются. Это позволяет вырезать произвольный по форме район из большого фрагмента карты.
- б) Выделение наибольшей компоненты связности: граф пешеходных путей рассматривается как мультиграф, сохраняются только пути, принадлежащие наибольшей компоненте. Изолированные фрагменты, не достижимые из основной сети, отбрасываются.
- в) Дедупликация точечных объектов: базовые точки, расположенные ближе 150 м друг к другу, и точки доставки, расположенные ближе 20 м, жадно прореживаются. При выборе среди близких кандидатов предпочтение отдаётся объектам с большим приоритетом (например, дарксторы предпочтительнее обычных супермаркетов, а главный вход предпочтительнее служебного).
- г) Разметка узких рёбер: для каждой вершины пешеходного пути вычисляется расстояние до ближайшего препятствия (ребра здания или ограждения), последовательность смежных вершин с зазором менее 3 м образует «узкий участок», путь помечается тегом `narrow=yes`, если хотя бы один такой участок имеет длину в пределах не менее 1 метра.
- д) Планаризация: ломаные разрезаются во всех точках, где они касаются или пересекают друг друга — в общих внутренних вершинах двух путей, Т-образных сочленениях (вершина одного пути попадает на ребро другого) и собственно пересечениях отрезков. После этого пути могут соприкасаться только конечными точками.

- е) Упрощение геометрии: каждая ломаная сводится к набору двухточечных отрезков с помощью алгоритма Рамера–Дугласа–Пекера (максимальное допустимое отклонение от исходной траектории — 5 м). Слишком короткие участки (менее 0.25 м) удаляются с последующим слиянием концов соседних рёбер в их середине.
- ж) Присоединение точек: каждая базовая точка и точка доставки проецируется на ближайший отрезок графа, если расстояние не превышает 20 м, в точке проекции отрезок разрезается, а от исходной точки до проекции добавляется ребро-коннектор. Коннекторы точек доставки, проходящие сквозь здания (более чем на половину своей длины), отбраковываются. Точки, которые не удалось подключить, исключаются из итогового графа.
- и) Устранение остаточных нарушений: после добавления коннекторов итеративно устраняются новые Т-образные сочленения и пересечения, возникающие вблизи точек подключения.
- к) Прореживание узких рёбер: в кластерах смежных узких отрезков жадно снимается пометка `narrow` с наиболее коротких, пока ни одна пара узких отрезков не разделяет общую вершину. Тем самым в итоговом графе узкие места представлены изолированными рёбрами.

Результатом работы утилиты является GeoJSON со всеми элементами итогового графа, который может быть напрямую использован для симуляции.

3.5 Используемые технологии

Основная часть проекта — симулятор — реализована на языке C++ стандарта C++20. Выбор языка продиктован требованиями к производительности: имитационные прогоны с тысячами агентов на крупных графах должны укладываться в разумное время. Утилита подготовки графов дорожной сети по выгрузкам OpenStreetMap, а также скрипты обработки результатов и построения отчётов написаны на языке Python 3.11.

В качестве системы сборки используется Bazel [20], который предоставляет удобное управление внешними зависимостями, а также обеспечивает герметичность и воспроизводимость сборки.

Из внешних библиотек C++ в проекте задействованы:

- Boost.Geometry [21] — вычисления элементарной вычислительной геометрии, геометрические индексы;
 - nlohmann/json [22] и PROJ [23] — разбор GeoJSON-описаний графов дорожной сети, преобразовании географических координат OpenStreetMap в локальную метрическую систему координат;
 - OpenCV [24] — покадровая отрисовка состояния симуляции при подготовке визуализаций;
 - CLI11 [25] — разбор аргументов командной строки исполняемых файлов симулятора;
 - Quill [26] — асинхронное логирование событий симуляции;
 - GoogleTest [27] — модульное тестирование компонент библиотеки.
- Структура основной части проекта представлена на рисунке 8.

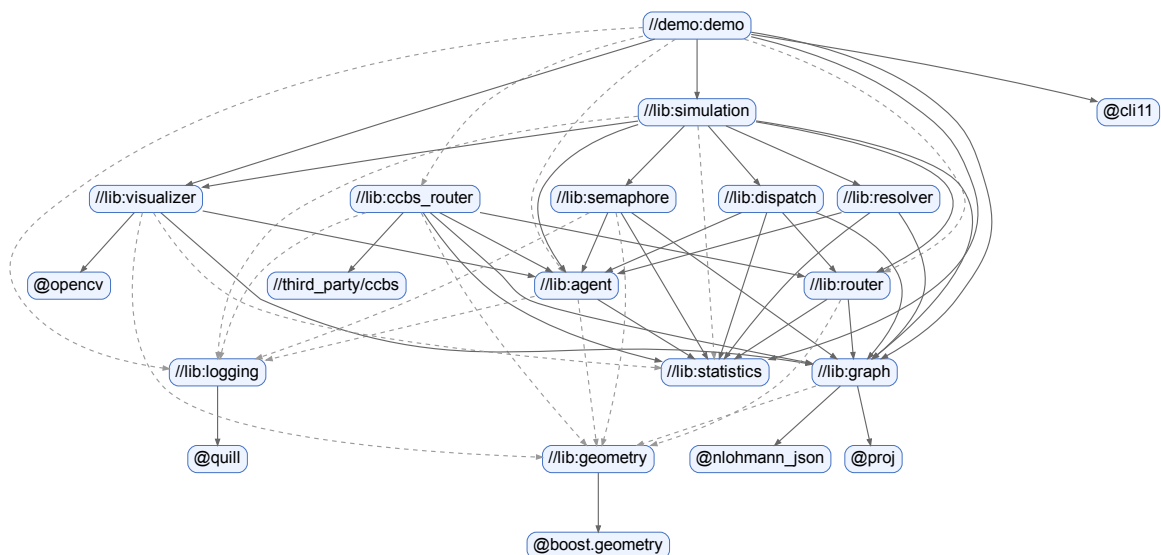


Рисунок 8 – Граф зависимостей

4 РЕЗУЛЬТАТЫ РАБОТЫ

4.1 Исходные данные

В качестве источника данных для графов дорожной сети выбраны три фрагмента карты Москвы разного размера. После процедуры обработки получены следующие графы.

Малый граф состоит из 75 ребер (10 узких) и 68 вершин (1 базовая точка и 8 точек доставки) и представлен на рисунке 9 (точки доставки обозначены зеленым цветом, базовые точки — красным, узкие ребра — желтой волнистой линией).

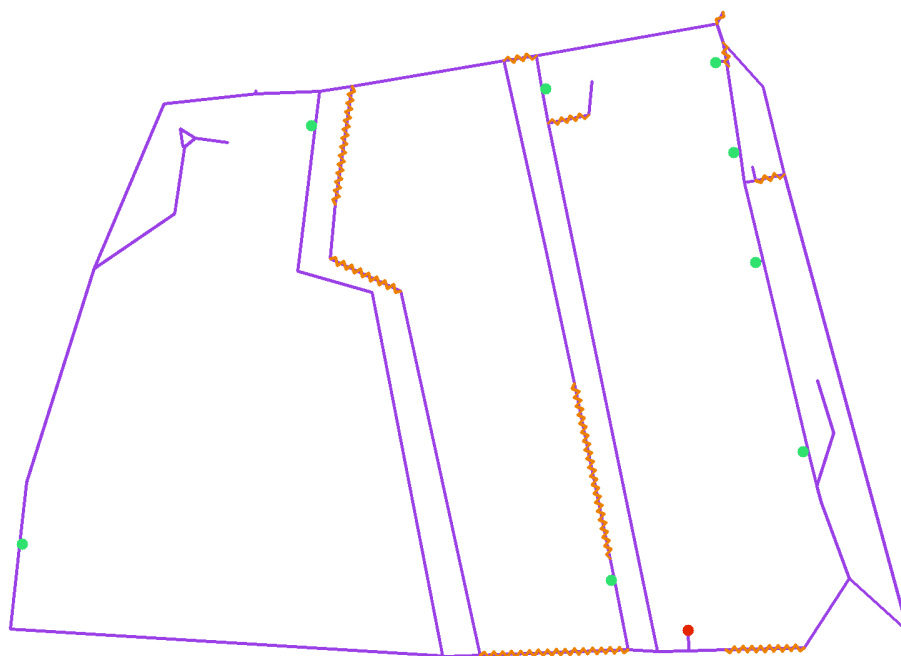


Рисунок 9 – Малый граф

Средний граф состоит из 1054 ребер (102 узких) и 887 вершин (6 базовых точек и 73 точек доставки).

Большой граф состоит из 3469 ребер (250 узких) и 2871 вершин (17 базовых точек и 264 точки доставки).

Средний и большой графы представлены в приложении А на рисунках А.1 и А.2, соответственно.

4.2 Используемые метрики

Сравнение методов проводится по интегральным характеристикам

потока заданий на фиксированном промежутке времени $[0, T]$, где T — длительность одного запуска симуляции. В ходе работы системы каждый агент $i \in A$ (2) многократно совершает циклы вида «базовая точка $\rightarrow$ точка доставки $\rightarrow$ базовая точка». Каждое перемещение агента по маршруту от одной из этих точек до следующей обозначим индексом ℓ . Обозначим через $\mathcal{R}(T)$ множество маршрутов, завершившихся в пределах $[0, T]$, а через $\mathcal{O}(T) \subseteq \mathcal{R}(T)$ — подмножество маршрутов, завершающих доставку заказа. Для каждого маршрута ℓ известны его длительность τ_ℓ и средняя скорость движения $\bar{v}_\ell = L_\ell / \tau_\ell$, где L_ℓ — пройденное агентом за маршрут расстояние.

В качестве основных показателей качества используются три величины:

- число выполненных заказов за время наблюдения:

$$N(T) = |\mathcal{O}(T)|; \quad (16)$$

- среднее время выполнения маршрута:

$$\bar{\tau}(T) = \frac{1}{|\mathcal{R}(T)|} \sum_{\ell \in \mathcal{R}(T)} \tau_\ell; \quad (17)$$

- средняя скорость движения агента по маршруту:

$$\bar{v}(T) = \frac{1}{|\mathcal{R}(T)|} \sum_{\ell \in \mathcal{R}(T)} \bar{v}_\ell. \quad (18)$$

Метрика $N(T)$ обобщает на продолжительный режим работы число достигнутых целей из классической MAPF, $\bar{\tau}(T)$ — метрику SOC (15), а $\bar{v}(T)$ характеризует эффективность исполнения индивидуальных маршрутов и снижается при возникновении заторов, ожиданий и отступлений.

Дополнительно фиксируется ряд вспомогательных метрик, имеющих смысл не для всех конфигураций и используемых для детального анализа работы отдельных механизмов:

- $\bar{\tau}_{\text{wait}}(T)$ — среднее время непрерывного пребывания агента в состоянии ожидания;
- $\bar{\tau}_{\text{rev}}(T)$ — среднее время непрерывного пребывания агента в состоянии отступления;
- $N_c(T)$ — суммарное число конфликтов, добавленных в $\mathcal{C}(t)$ за время

наблюдения (в ходе разрешения конфликтов методом управления разездом).

4.3 Проверка статистической значимости

Поскольку как порядок поступления заказов, так и начальное распределение агентов по графу разыгрываются псевдослучайно, каждая из метрик (16)–(18) при фиксированной конфигурации графа, числа агентов и метода разрешения конфликтов представляет собой случайную величину. Различие средних значений метрики между двумя методами, наблюдаемое на конечном числе запусков, может быть вызвано как действительным превосходством одного из них, так и случайным разбросом входных условий. Чтобы отделить систематический эффект от шума, а заодно убедиться в том, что наблюдаемые различия имеют практический смысл, а не только формально статистический, используем методы проверки статистической значимости. Сравнение двух методов сводится при этом к сравнению двух независимых выборок, составленных из значений метрики, полученных при различных запусках симулятора. Ниже описаны методы, которые будут применяться для такого сравнения.

В качестве основного критерия используется U-критерий Манна–Уитни [28] (в реализации `scipy.stats.mannwhitneyu`). Нулевая гипотеза H_0 состоит в том, что распределения X и Y совпадают, альтернатива H_1 — в том, что одна из выборок стохастически доминирует над другой. При $p < \alpha$ (используется уровень значимости $\alpha = 0.05$) гипотеза H_0 отвергается.

Выбор именно этого критерия обусловлен следующими соображениями. Во-первых, распределения исследуемых метрик заметно отличаются от нормального: $N(T)$ является дискретной счётной величиной, а $\bar{\tau}(T)$ и $\bar{v}(T)$ — усреднениями по множеству маршрутов с тяжёлыми хвостами, возникающими при возникновении заторов. Это делает применение t -критерия Стьюдента некорректным. Во-вторых, критерий Манна–Уитни не требует предположений о форме распределения и устойчив к выбросам, что важно при сравнении методов, у которых отдельный неудачный прогон может дать аномально большое значение $\bar{\tau}(T)$. При множественных попарных сравнениях для контроля ложноположительных результатов используется пошаговая поправка Холма [29] (в реализации `statsmodels.stats.multitest.multipletests`).

Чтобы дополнить p -value оценкой практической значимости, совместно с критерием Манна–Уитни вычисляется дельта Клиффа [30]. Для качественной интерпретации используется шкала Романо [31]: эффект считается пренебрежимо малым при $|\delta| < 0,147$, малым при $|\delta| < 0,33$, средним при $|\delta| < 0,474$ и большим в остальных случаях.

Два метода считаются практически различимыми по выбранной метрике, если одновременно выполняются условия $p^{\text{holm}} < 0,05$ и $|\delta| \geq 0,147$. Случай $p^{\text{holm}} < 0,05$ при пренебрежимо малом $|\delta|$ интерпретируется как статистический артефакт большого объема выборки и во внимание не принимается; случай $p^{\text{holm}} \geq 0,05$ означает, что наблюдаемых данных недостаточно для отклонения гипотезы о совпадении распределений.

4.4 Результаты работы

Каждый запуск симулятора определяется четырьмя факторами: графом дорожной сети G , числом агентов $|A|$, методом разрешения конфликтов и используемым планировщиком маршрутов. Длительность симуляции ограничена 12 часами (43200 секунд) виртуального времени. Для набора экспериментальной выборки каждая конфигурация запускалась несколько раз (но не более двухсот) с суммарным ограничением по времени работы в 5 минут (10 минут для тяжелых конфигураций).

В сравнении участвуют:

- графы трех размеров, указанные в раздел 4.1;
- конфигурации агентов в размерах 10, 50, 100, 500, 1000 штук;
- методы разрешения конфликтов:
 - управление разъездом (далее «разъезд»),
 - глобальные ограничения через механизм семафоров (далее «семафор»);
- планировщики маршрутов:
 - классический A^* ,
 - адаптивный A^* ,
 - CCBS (используется без механизма разрешения конфликтов).

Поскольку абсолютные значения метрик (16)–(18) изменяются на порядки между парами $(G, |A|)$, объединение результатов симуляции из разных пар в одну выборку маскирует сравниваемый эффект под влияние

масштаба. Поэтому каждая пара $(G, |A|)$ рассматривается как самостоятельная страта: все попарные сравнения и поправка Холма применяются только внутри страты, а итоговый вывод получается агрегированием результатов по всем доступным стратам.

Основным объектом сравнения является метод разрешения конфликтов, а планировщик играет вспомогательную роль. В связи с этим анализ результатов разбит на три шага:

- а) для каждого метода разрешения конфликтов внутри страты выбирается планировщик, с которым этот метод показывает лучший результат;
- б) методы разрешения конфликтов сравниваются между собой в их наилучших конфигурациях;
- в) полученные конфигурации сравниваются с CCBS.

4.4.1 Выбор метода планирования для методов разрешения конфликтов

Для каждого метода разрешения конфликтов R и каждой страты $(G, |A|)$ сравниваются две выборки: R в паре с классическим A^* против R в паре с адаптивным A^* , по каждой из трех основных метрик (16)–(18).

По каждой метрике голос отдается в пользу того планировщика, для которого метрика лучше в нужном направлении (число заказов и средняя скорость — больше, время заказа — меньше), при условии что $p^{\text{holm}} < 0,05$ и $|\delta| \geq 0,147$. В остальных случаях голос засчитывается как ничейный. Побеждает планировщик, получивший хотя бы два голоса из трех; при делении 1–1–1 приоритет имеет метрика $N(T)$; при трех ничьих победитель не определен.

В таблице 1 представлен фрагмент сравнения методов планирования для метода «разъезд» по метрике средней скорости.

Таблица 1 – Сравнение методов планирования для метода «разъезд» по метрике средней скорости

Граф	Число агентов	медиана		δ	p^{holm}	результат
		A*	адапт. A*			
мал.	10	2.063	2.069	−0.927	$9.882 \cdot -39$	адапт. A*
	100	1.934	1.993	−1	$1.574 \cdot -28$	адапт. A*
	1000	1.575	1.944	−1	$7.589 \cdot -8$	адапт. A*
ср.	10	2.137	2.142	−0.893	$5.284 \cdot -36$	адапт. A*
	100	2.099	2.117	−1	$2.583 \cdot -24$	адапт. A*
	1000	1.976	2.072	−1	0.00199	адапт. A*
бол.	10	2.116	2.138	−0.999	$9.198 \cdot -45$	адапт. A*
	100	2.093	2.146	−1	$4.232 \cdot -24$	адапт. A*
	1000	2.031	2.146	−1	0.0035	адапт. A*

Из таблицы видно, что при небольшом числе агентов вклад метода планирования практически незаметен, что обусловлено меньшим числом конфликтов в таких конфигурациях и соответственно меньшим эффектом от разрешения конфликтов в целом. При увеличении числа агентов начинают играть бóльшую роль физические ограничения локации, а также меньший объем данных, который был собран в таких запусках. Адаптивный A* показывает себя лучше, чем неадаптивный.

В таблице 2 представлено сравнение методов планирования для метода «семафор» по метрике числа заказов. Выводы из таблицы можно сделать аналогичные.

Таблица 2 – Сравнение методов планирования для метода «семафор» по метрике числа заказов

Граф	Число агентов	медиана		δ	p^{holm}	результат
		A*	адапт. A*			
мал.	10	2064.5	2067	−0.0574	0.836	ничья
	100	18081	18656	−1	$6.507 \cdot 10^{-30}$	адапт. A*
	1000	66273	89888	−1	$1.991 \cdot 10^{-11}$	адапт. A*
ср.	10	606	608.5	−0.07365	0.585	ничья
	100	5942	5973.5	−0.508122	$7.366 \cdot 10^{-6}$	адапт. A*
	1000	51529	54821	−1	$6.625 \cdot 10^{-5}$	адапт. A*
бол.	10	309.5	312	−0.1913	0.0138	адапт. A*
	100	3115	3184	−0.975684	$4.289 \cdot 10^{-18}$	адапт. A*
	1000	28104	31569	−1	0.00024	адапт. A*

При агрегировании результата по всем метрикам получают следующие результаты, которые представлены на рисунке 10 (цвет ячеек отражает дельту Клиффа по метрике средней скорости, статистическая значимость выражена звездочками).

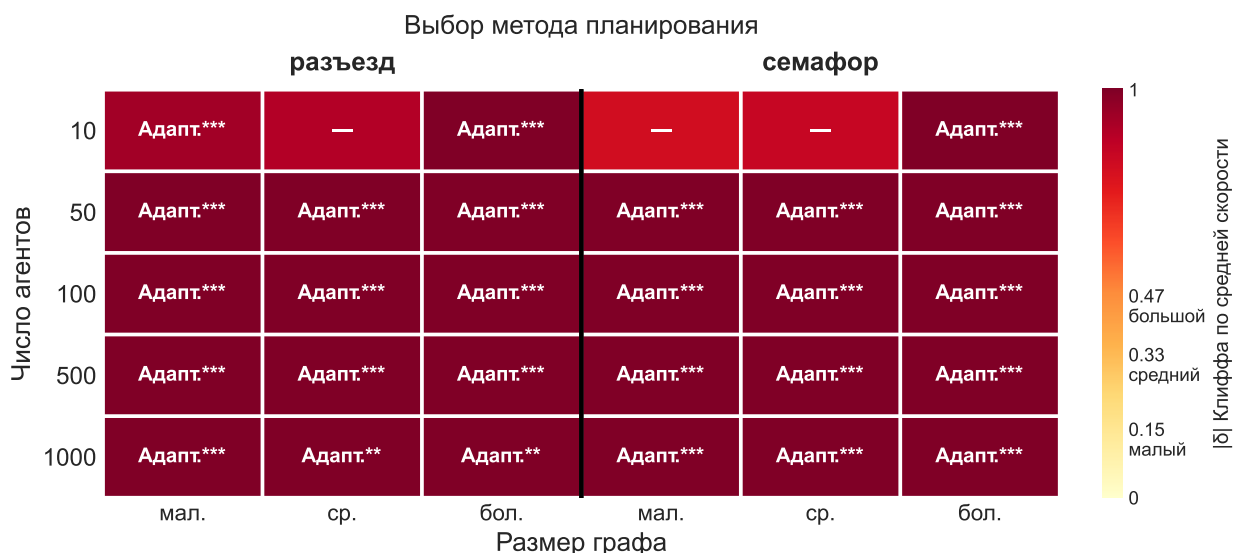


Рисунок 10 – Итоговые результаты выбора метода планирования для методов разрешения конфликтов

Таким образом, при небольшом числе агентов может быть использован любой метода планирования, но в целом адаптивный A* показывает себя лучше в большинстве сценариев и ни в одном сценарии не показывает себя хуже. Для

всех методов разрешения конфликтов выбираем адаптивный A^* .

4.4.2 Сравнение методов разрешения конфликтов

В каждой страте $(G, |A|)$ метод «разъезд» в выбранной для него конфигурации сравнивается с методом «семафор» в выбранной для него конфигурации.

Каждой паре «метод $\times$ метрика» внутри страты сопоставляется голос по тому же правилу, что и в предыдущем шаге. Победитель страты определяется большинством голосов по всем метрикам и всем парам разложения.

В таблице 3 представлено сравнение методов разрешения конфликтов по метрике среднего времени заказа.

Таблица 3 – Сравнение методов разрешения конфликтов по метрике среднего времени заказа

Граф	Число агентов	медиана		δ	p^{holm}	результат
		«семафор»	«разъезд»			
мал.	10	107	104.4	0.964	$7.112 \cdot 10^{-32}$	«семафор»
	50	113.5	110.3	1	$7.64 \cdot 10^{-34}$	«семафор»
	100	117	115.8	0.946	$5.505 \cdot 10^{-25}$	«семафор»
	500	146.9	176.9	-1	$8.479 \cdot 10^{-14}$	«разъезд»
	1000	168	239.3	-1	$8.392 \cdot 10^{-07}$	«разъезд»
ср.	10	355.2	354.7	0.063	0.8903	ничья
	50	363.3	359.5	0.619	$7.847 \cdot 10^{-14}$	«семафор»
	100	366.4	361.2	0.932	$1.244 \cdot 10^{-18}$	«семафор»
	500	381.7	376.9	1	$1.22 \cdot 10^{-08}$	«семафор»
	1000	393.3	393.6	-0.367	0.527	ничья
бол.	10	687.3	690.7	-0.108	0.5672	ничья
	50	677.4	678.2	-0.05	1	ничья
	100	680.3	677	0.31	0.00363	«семафор»
	500	684.6	681.3	0.721	$2.382 \cdot 10^{-05}$	«семафор»
	1000	687.5	682.8	0.873	0.00419	«семафор»

Итоговые результаты по всем метрикам представлены на рисунке 11 (цвет ячеек отражает дельту Клиффа по метрике средней скорости).

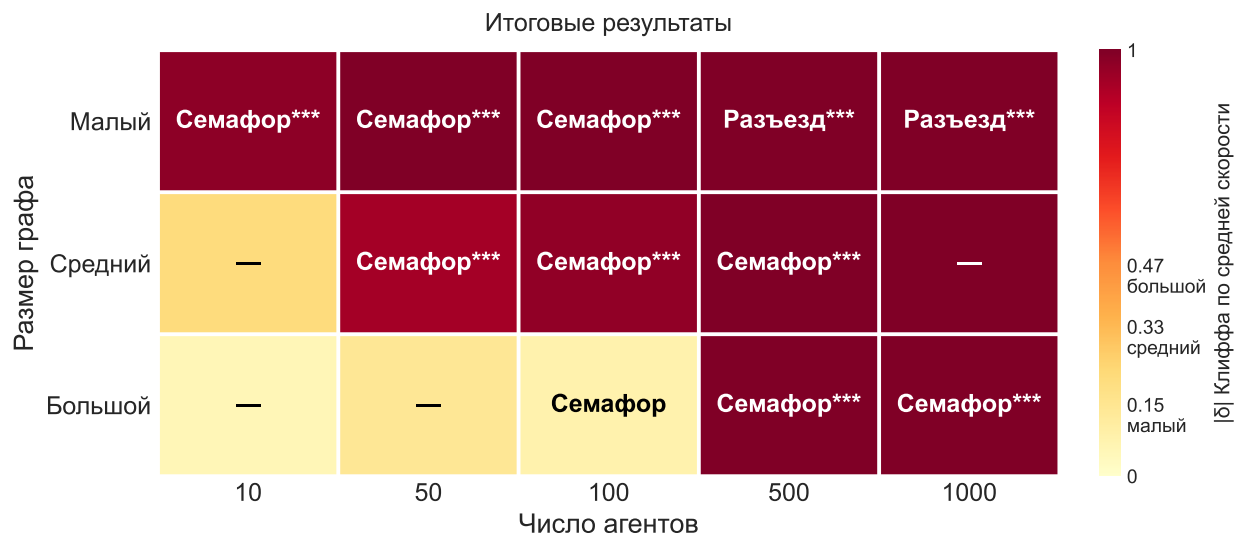


Рисунок 11 – Итоговые результаты выбора метода разрешения конфликтов

По результатам этого сравнения видно, что при небольшом числе агентов относительно размера локации выбор метода разрешения конфликтов не имеет большой разницы из-за малого числа самих конфликтов. В большинстве сценариев лучше себя показывает метод «семафор», за исключением случаев с большим числом агентов на меньших графах, в которых он уступает методу разъездов.

4.4.3 Сравнение с CCBS

Для каждой страты $(G, |A|)$ с $|A| \in \{10, 50, 100, 500, 1000\}$ строятся две пары сравнения: CCBS против «разъезда» в его наилучшей конфигурации и CCBS против «семафора» в его наилучшей конфигурации, все шесть тестов одной страты объединяются в одно семейство для поправки Холма.

В таблице 4 представлен небольшой фрагмент сравнения методов разрешения конфликтов с CCBS.

Таблица 4 – Сравнение методов разрешения конфликтов с CCBS (фрагмент)

Граф	$ A $	метод	метрика	δ	p^{holm}	результат
мал.	10	«разъезд»	число заказов	-0.898	$1.466 \cdot 10^{-36}$	«разъезд»
		«семафор»	среднее время	1	$1.464 \cdot 10^{-44}$	«семафор»
ср.	100	«разъезд»	число заказов	-0.089	1	ничья
		«разъезд»	среднее время	0.079	1	ничья
		«семафор»	средняя скорость	-1	$5.318 \cdot 10^{-11}$	«семафор»
бол.	1000	«разъезд»	число заказов	0.534	0.1385	ничья
		«семафор»	среднее время	0.576	0.1336	ничья
		«семафор»	средняя скорость	0.919	0.003145	CCBS

Итоговые результаты по всем метрикам представлены на рисунке 12 (цвет ячеек отражает дельту Клиффа по метрике числа заказов).

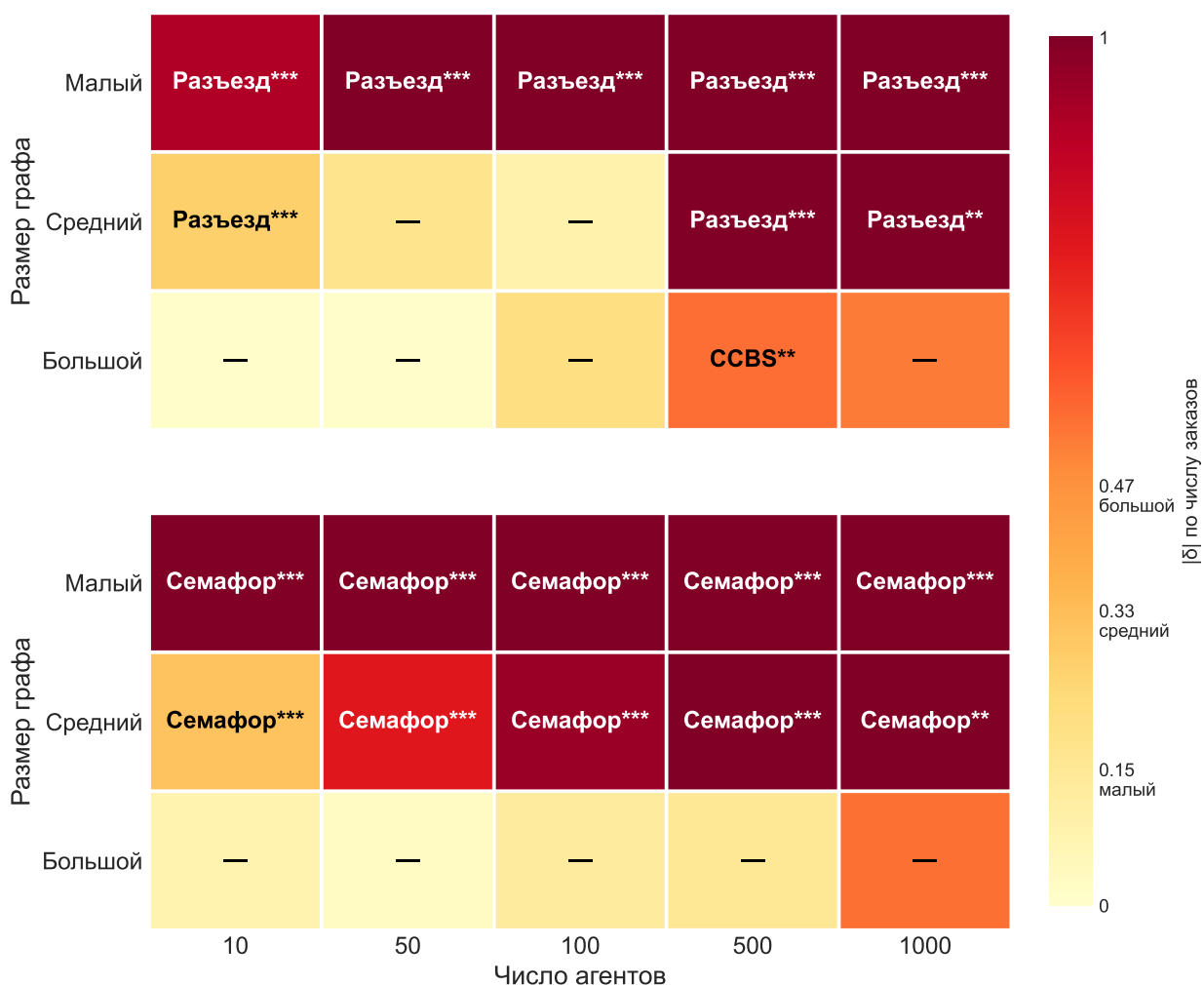


Рисунок 12 – Итоговые результаты сравнения методов разрешения конфликтов с CCBS

По результатам итогового сравнения видно, что методы разрешения конфликтов показывают себя не хуже, чем метод многоагентного планирования. В большинстве случаев эти методы превосходят CCBS, но начинают уступать при увеличении размера задачи. Однако стоит отметить, что при увеличении размера задачи начинает испытывать трудности и метод CCBS, что выражается в его скорости работы и сходимости.

Основные выводы из анализа результатов такие, что методы локального разрешения конфликтов могут быть использованы для управления автономными агентами и не уступают методам многоагентного планирования по качеству. Вместе с тем, методам разрешения конфликтов очень помогает использование информации о состоянии среды, что может быть выражено в использовании адаптивного одногоагентного метода построения маршрутов.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы магистра была рассмотрена задача глобального планирования движения группы автономных доставочных агентов по взвешенному графу дорожной сети при наличии узких мест, на которых одновременное встречное движение физически невозможно. Сформулирована формальная постановка задачи, обобщающая классическую MAPF на непрерывное время и продолжительный режим работы, а также введён набор интегральных метрик, пригодных для сравнения методов управления.

Основное внимание уделено противопоставлению двух принципиально различных подходов к работе с узкими местами: оптимальному многоагентному планированию с учётом конфликтов на этапе поиска (на примере алгоритма CCBS) и более простым методам непосредственного разрешения конфликтов на этапе исполнения. В рамках второго подхода были предложены и реализованы две стратегии — управление разъездом агентов и предотвращение конфликтов на основе системы семафоров, — которые дополнены адаптивной модификацией A^* , использующей накапливаемые в ходе симуляции наблюдения за скоростью прохождения рёбер в качестве динамических весов.

Большую часть работы заняла разработка экспериментальной среды. Реализован модульный симулятор, в котором подсистемы планирования и разрешения конфликтов выделены в подключаемые компоненты с единым интерфейсом, что позволяет запускать произвольные комбинации алгоритмов на одних и тех же исходных данных.

Сравнительный эксперимент проводился на графах, полученных из фрагментов реальной городской карты. Сравнение методов разрешения конфликтов с CCBS подтвердило основную гипотезу работы: простые методы разрешения конфликтов в сочетании с адаптивным методом построения маршрута не уступают полному многоагентному планированию, а в сценариях малого и среднего размера превосходят его по всем интегральным метрикам. При увеличении размера задачи CCBS начинает выигрывать по качеству, но одновременно испытывает существенные трудности с временем работы и сходимостью.

Таким образом, поставленные в работе задачи решены полностью:

построена формальная модель задачи, рассмотрены существующие и предложены альтернативные методы её решения, разработана воспроизводимая программная среда, подготовлен репрезентативный набор сценариев и проведено их сравнительное исследование с оценкой статистической значимости. Основной практический вывод состоит в том, что для задач автономной доставки на последней миле перенос согласования движения с этапа планирования на этап исполнения является обоснованной альтернативой полному многоагентному планированию: при существенно меньших вычислительных затратах такой подход обеспечивает сопоставимое, а то и более высокое качество совместного движения, что делает его перспективным для применения в реальных системах управления группами автономных агентов.

Дальнейшее развитие работы возможно в нескольких направлениях. Во-первых, в расширении модели среды учётом других участников дорожного движения и неоднородности самих агентов. Во-вторых, в исследовании гибридных схем, в которых многоагентное планирование применяется выборочно — только к подмножеству агентов, чьи маршруты пересекаются в наиболее загруженных узких местах, — а остальные согласуются предложенными лёгкими методами.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Autonomous last-mile delivery robots: a literature review / E. Alverhed [и др.] // European Transport Research Review. — 2024. — Т. 16, № 1.
2. Hybrid Classical/RL Local Planner for Ground Robot Navigation / V. D. Sharma [и др.]. — 2024.
3. Mobile Robot Path Planning in Dynamic Environments: A Survey / K. Cai [и др.] // CoRR. — 2020.
4. Wayformer: Motion Forecasting via Simple & Efficient Attention Networks / N. Nayakanti [и др.]. — 2022.
5. Multi-Agent Pathfinding: Definitions, Variants, and Benchmarks / R. Stern [и др.] // Proceedings of the International Symposium on Combinatorial Search. — 2019. — Т. 10, № 1. — С. 151—158.
6. Multi-Agent Pathfinding with Continuous Time / A. Andreychuk [и др.] // Artificial Intelligence. — 2022. — Т. 305. — С. 103662.
7. Lifelong Multi-Agent Path Finding in Large-Scale Warehouses / J. Li [и др.] // Proceedings of the AAAI Conference on Artificial Intelligence. — 2021. — Т. 35, № 13. — С. 11272—11281.
8. Silver D. Cooperative Pathfinding // Artificial Intelligence and Interactive Digital Entertainment Conference. — 2005.
9. Phillips M., Likhachev M. SIPP: Safe interval path planning for dynamic environments // Proceedings - IEEE International Conference on Robotics and Automation. — 2011. — С. 5628—5635.
10. Okumura K. LaCAM: Search-Based Algorithm for Quick Multi-Agent Pathfinding // Proceedings of the AAAI Conference on Artificial Intelligence. — 2023. — Т. 37, № 10. — С. 11655—11662.
11. Okumura K. Improving LaCAM for Scalable Eventually Optimal Multi-Agent Pathfinding // Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, (IJCAI-23). — 2023. — С. 243—251.
12. Okumura K. Engineering LaCAM*: Towards Real-Time, Large-Scale, and Near-Optimal Multi-Agent Pathfinding // Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems, (AAMAS 2024). — 2024.

13. Priority inheritance with backtracking for iterative multi-agent path finding / K. Okumura [и др.] // Artificial Intelligence. — 2022. — Т. 310.
14. Hart P. E., Nilsson N. J., Raphael B. A Formal Basis for the Heuristic Determination of Minimum Cost Paths // IEEE Transactions on Systems Science and Cybernetics. — 1968. — Т. 4, № 2. — С. 100—107.
15. The on-line shortest path problem under partial monitoring / A. Gyorgy [и др.]. — 2007.
16. Continuous-CBS. — URL: <https://github.com/PathPlanning/Continuous-CBS> (дата обращения 15.04.2026).
17. GeoJSON specification. — URL: <https://geojson.org/> (дата обращения 15.04.2026).
18. FFmpeg. — URL: <https://ffmpeg.org/> (дата обращения 15.04.2026).
19. OpenStreetMap. — URL: <https://www.openstreetmap.org> (дата обращения 15.04.2026).
20. Bazel. — URL: <https://bazel.build/> (дата обращения 15.04.2026).
21. Boost.Geometry. — URL: <https://www.boost.org/library/latest/geometry/> (дата обращения 15.04.2026).
22. JSON for Modern C++. — URL: <https://json.nlohmann.me/> (дата обращения 15.04.2026).
23. PROJ. — URL: <https://proj.org/> (дата обращения 15.04.2026).
24. OpenCV. — URL: <https://opencv.org/> (дата обращения 15.04.2026).
25. CLI11. — URL: <https://github.com/cliutils/cli11> (дата обращения 15.04.2026).
26. Quill. — URL: <https://github.com/odygrd/quill> (дата обращения 15.04.2026).
27. GoogleTest. — URL: <https://github.com/google/googletest> (дата обращения 15.04.2026).
28. Mann H. B., Whitney D. R. On a test of whether one of two random variables is stochastically larger than the other // The annals of mathematical statistics. — 1947. — С. 50—60.

29. Holm S. A Simple Sequentially Rejective Multiple Test Procedure // Scandinavian Journal of Statistics. — 1979. — T. 6, № 2. — С. 65—70.
30. Cliff N. Dominance statistics: Ordinal analyses to answer ordinal questions. — 1993.
31. Romano J., Kromrey J. Appropriate Statistics for Ordinal Level Data: Should We Really Be Using t-test and Cohen's d for Evaluating Group Differences on the NSSE and other Surveys? — 2006.

ПРИЛОЖЕНИЕ А

Графы дорожной сети

Средний и большой графы дорожной сети представлены на рисунках А.1 и А.2, соответственно (точки доставки обозначены зеленым цветом, базовые точки — красным, узкие ребра — желтой волнистой линией).

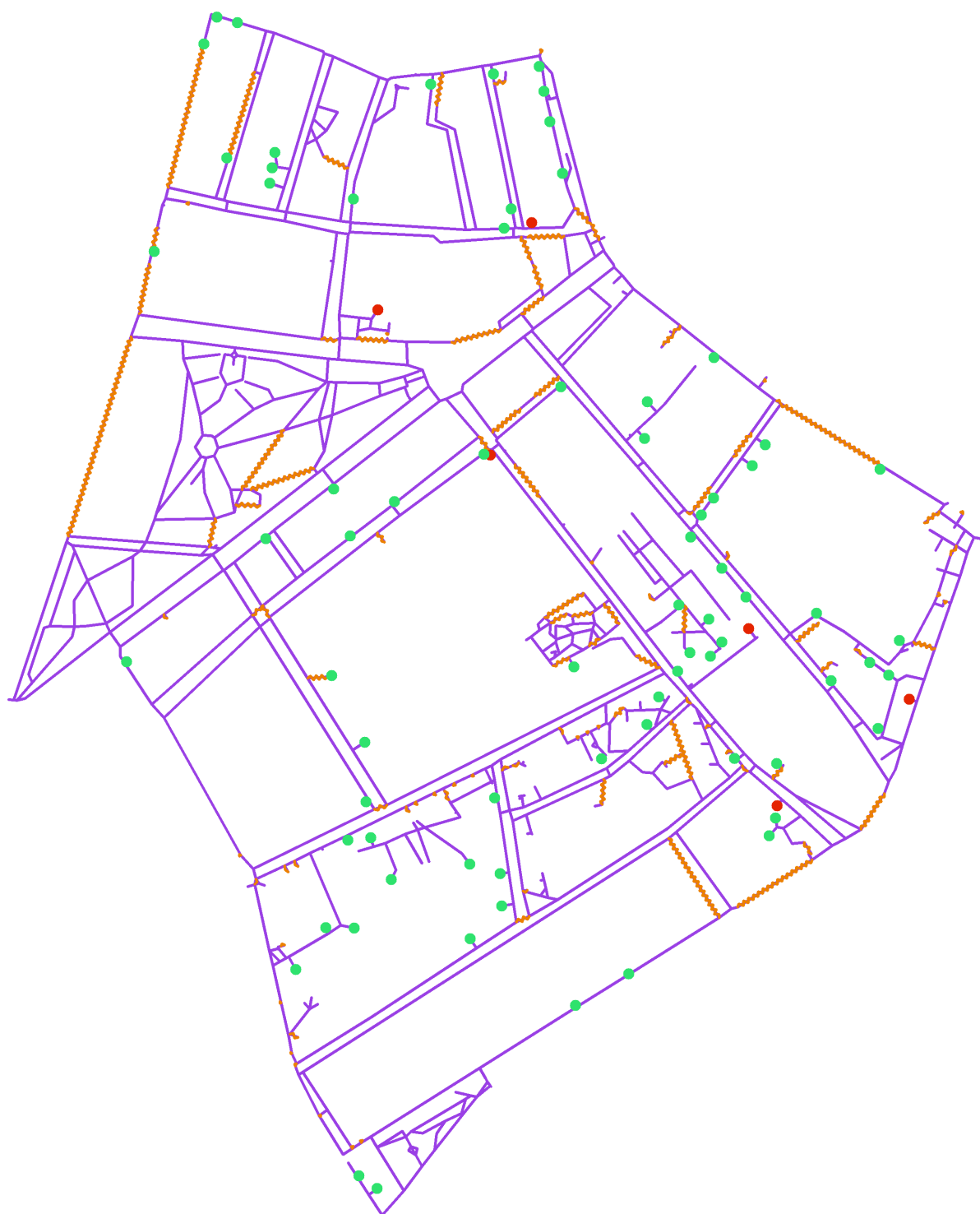


Рисунок А.1 – Средний граф

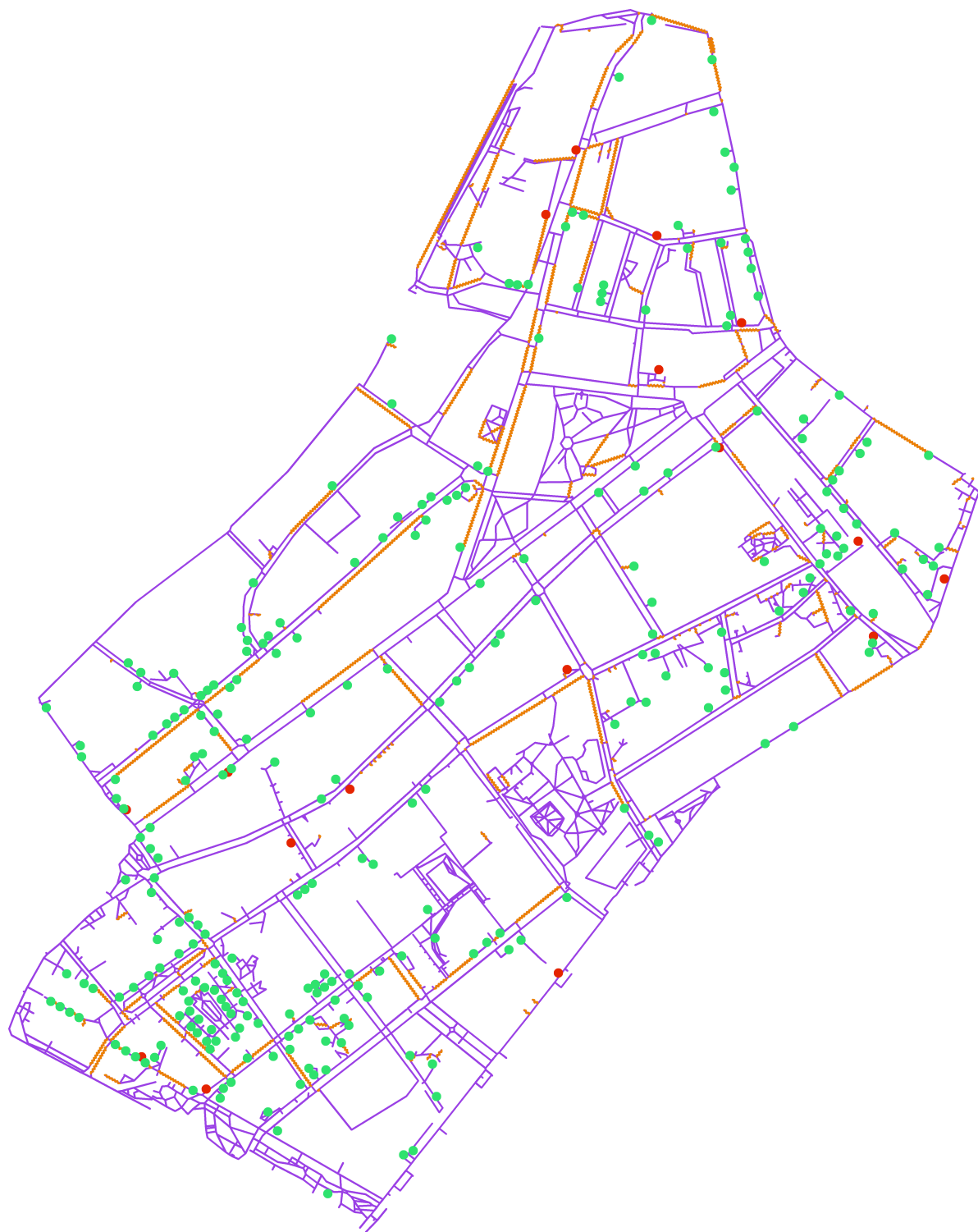


Рисунок А.2 – Большой граф

ПРИЛОЖЕНИЕ Б

Исходный код

Репозиторий с исходным кодом расположен по адресу <https://github.com/iktovr/master-diploma>. Ссылка в формате QR-кода представлена на рисунке Б.1.



Рисунок Б.1 – QR-код со ссылкой на репозиторий